American International University-Bangladesh (AIUB)
Faculty of Science and Technology

Design of a Machine Learning-Based Smart Health Assistant for Early Diagnosis and Specialist Mapping

Iftakhar Ahmed Rakib (22-46015-1)
Sharif Ahmmed Nazmuzzaman (20-41939-1)
Arman Al Arefin (22-46027-1)
Golam Rabbani (22-46560-1)

A Thesis submitted for the degree of Bachelor of Science (BSc) in
Computer Science and Engineering (CSE) at
American International University Bangladesh (AIUB
Faculty of Science and Technology (FST)

Spring 2025-2026 Semester
Submission Date: July 23, 2026

Declaration

This thesis is composed of our original work, and contains no material previously published or written by another person except where due reference has been made in the text. We have clearly stated the contribution of others to our thesis as a whole, including statistical assistance, survey design, data analysis, significant technical procedures, professional editorial advice, financial and any other original research work used or reported in our thesis. The content of our thesis is the result of work we have carried out since the commencement of the Thesis.

We acknowledge that copyright of all material contained in my thesis resides with the copyright holder(s) of that material. Where appropriate we have obtained copyright permission from the copyright holder to reproduce material in this thesis and have sought permission from co-authors for any jointly authored works included in the thesis.

Iftakhar Ahmed Rakib
22-46015-1
Department: CSE

Sharif Ahmmed Nazmuzzaman
20-41939-1
Department: CSE

Arman Al Arefin
22-46027-1
Department: CSE

Golam Rabbani
22-46560-1
Department: CSE

Approval

This thesis titled "**Design of a Machine Learning-Based Smart Health Assistant for Early Diagnosis and Specialist Mapping**" has been submitted to the following respected members of the Board of Examiners of the Department of Computer Science in partial fulfilment of the requirements for the degree of Bachelor of Science in Computer Science on August 6, 2026, and has been accepted as satisfactory.

Syeda Anika Tasnim
Assistant Professor
Department of Computer Science
American International University-Bangladesh

MOHAIMEN- BIN- NOOR
Assistant Professor & External
Department of Computer Science
American International University-Bangladesh

Prof. Dr. Muhammad Firoz Mridha
Professor & Head (UG)
Department of Computer Science
American International University-Bangladesh

Prof. Dr. Khandakar Tabin Hasan
Professor & Associate Dean
Faculty of Science and Technology
American International University-Bangladesh

Prof. Dr. Dip Nandi
Professor & Dean
Faculty of Science and Technology
American International University-Bangladesh

Acknowledgement

We would like to express our sincere gratitude to our supervisor, Syeda Anika Tasnim, **[Assistant Professor, Department of Computer Science]**, for her guidance, patience, and valuable feedback throughout every stage of this research, from the initial selection of the topic to the final review of this thesis. Her advice on structuring the comparative model evaluation and framing the system's real-world limitations shaped this work in ways we could not have managed alone.

We are also grateful to each of our team members for their dedication and collaboration throughout the design, development, and deployment of the Smart Health Assistant application. Each of us contributed a distinct part of this system, and its completion would not have been possible without that shared effort.

We would like to thank American International University-Bangladesh for providing the academic environment and resources that made this research possible.

Finally, we are thankful to our families and friends for their constant encouragement and support throughout this project.

Table of Content

List of Figures

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Full form
AI	Artificial Intelligence
ML	Machine Learning
DL	Deep Learning
MLP	Multilayer Perceptron
LR	Logistic Regression
RF	Random Forest
XGBoost	Extreme Gradient Boosting
LightGBM	Light Gradient Boosting Machine
CV	Cross-Validation
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
UI/UX	User Interface / User Experience
CORS	Cross-Origin Resource Sharing
SQLite	Structured Query Language Lite (embedded database)
F1	F1-score (harmonic mean of precision and recall)
PRISMA	Preferred Reporting Items for Systematic Reviews and Meta-Analyses
GPU	Graphics Processing Unit
CPU	Central Processing Unit
HTTP/HTTPS	Hypertext Transfer Protocol (Secure)

Abstract

Diagnosing disease from self-reported symptoms remains a persistent gap between formal clinical care and the moment a patient first notices something is wrong. Most existing symptom-checking tools are built around a narrow disease domain, a single dataset, or an evaluation method that reports one aggregate accuracy figure without examining how the model actually behaves across hundreds of possible conditions. This thesis set out to close that gap by building a machine learning-based smart health assistant that predicts likely diseases from user-reported symptoms, evaluating candidate models across two independently constructed datasets rather than relying on a single data source.

The first dataset, a Master Dataset, was built by merging several smaller publicly available symptom-disease datasets into one internally consistent resource, requiring the standardisation of disease names and symptom terminology across sources that had originally used inconsistent labels. On this dataset, a multinomial Logistic Regression model and a Random Forest classifier were compared under an identical train-test split, and Logistic Regression was selected for deployment. The second dataset, a much larger single-source collection of over 200,000 records, was used to train and compare four further candidates, XGBoost, LightGBM, TabNet, and a multilayer perceptron, under a shared preprocessing and evaluation protocol, with every model's stability additionally confirmed through 5-fold stratified cross-validation.

On the Master Dataset, Logistic Regression reached 78.45% top-1 accuracy and 91.10% top-3 accuracy, outperforming Random Forest (75.44%/88.88%), and was selected for deployment on the strength of its accuracy, calibration, and lightweight inference cost. On the larger dataset, the multilayer perceptron and TabNet emerged as the strongest and most consistently validated candidates, with cross-validated means of 83.01%/94.81% and 81.57%/93.99% respectively, ahead of XGBoost (80.17%/93.58% cross-validated) and LightGBM (64.63%/88.54% cross-validated). Benchmarked against five previously published symptom-disease prediction studies, this thesis's results did not achieve the single highest reported accuracy, but were shown to be more rigorously and transparently evaluated than any of the five external comparators, each of which omitted macro-averaged metrics, confusion matrix analysis, shortlist-based accuracy, or cross-validated stability reporting.

The selected Logistic Regression model was deployed inside a Flutter-based mobile

application, backed by a FastAPI service, that returns a ranked shortlist of candidate diseases enriched with descriptions, precautions, severity indicators, and specialist recommendations, framed explicitly as a starting point for discussion with a healthcare professional rather than as a diagnosis. Taken together, these findings suggest that combining a carefully unified, deployment-tested dataset with a broader, cross-validated comparative study on a larger corpus provides both a working, honestly evaluated triage tool and useful evidence for how model choice should adapt to dataset scale and deployment constraints in future health-assistant systems.

Keywords

disease prediction, symptom checker, machine learning, logistic regression, tabnet, cross-validation, healthcare triage, top-3 accuracy, specialist recommendation, mobile health application

I. Introduction

A. Introduction

Artificial intelligence has come a long way in the last ten years: from an interesting novelty in medical analysis to one of the key pillars of digital-health research. For the first decade of its use, machine learning algorithms were applied to tightly controlled domains, where the input data was largely structured and clear, i.e., laboratory tests and images interpreted by clinicians [6], [10], [15], [31]. However, the tendency in recent years shows that machine learning is now used in a broader sense – integrated into the real-life clinical workflow assisting doctors and nurses in their day-to-day business [1], [3], [5]–6, [10]–12, [15]–16, [18], [21], [24], [26]–27, [30]–31, [36], [43], [46], [48]–49, [55]–56, [61]. Why is this important? Because in most cases, the interaction with health-related information begins long before the consultation with medical professionals – when the individual trying to understand their health state better needs guidance and support.

In fact, this trend reflects a broader tendency – the digitization of the healthcare ecosystem. Increasingly, the responsibility for managing one's health falls on the individual, rather than clinicians. With self-tracking becoming more common, people turn to healthcare information online before consulting a doctor. They need help making sense of the information at hand in order to understand their health state and decide what to do next. This is where machine learning becomes useful – as an opportunity to recognize patterns that could be related to diseases and turn them into actionable insights [6], [10], [15], [26]–27, [31], [36], [43], [48]–49, [55]–59, [61].

Amongst the possible applications of machine learning to healthcare, disease prediction systems seem to be the most developed and promising. There is substantial literature on the application of machine learning to diagnostic support, both in structured and unstructured settings [1], [5]. In particular, there are numerous studies demonstrating that where structured data is concerned (lab tests, imaging, selected clinical variables), machine

learning often achieves excellent diagnostic accuracy and precision [1], [5]. Additionally, considerable research has been done into predicting specific diseases, including chronic conditions, cardiological syndromes, headaches, and Alzheimer's – and the literature is full of relevant examples demonstrating how machine learning can support diagnosis in a narrower domain [2], [4], [7]–8, [25], [37]. Taken together, these examples show that in terms of diagnostic support, machine learning can indeed achieve high diagnostic accuracy and precision.

However, as soon as the focus turns to the average individual trying to understand their condition before consulting a doctor, the setting changes fundamentally – as most people cannot rely on having structured data pointing to one condition. Instead, one must rely on symptoms – and the ability to reason from symptoms to possible diseases. Symptom-based reasoning is a much broader task, covering not only diagnostic support but including collaborative health dialogues and symptom-checking [3], [16]–17, [51]–53. Symptom-based diagnosis is a much more complex task, and the literature shows that there are significant practical challenges associated with building such a system, including the frequent ambiguity of symptoms pointing to multiple conditions [16]. Symptom-based diagnosis presents a major challenge, and the current literature addresses it in a number of ways, including building collaborative reasoning systems [3], [16]–17, [51]–53.

Additionally, it is worth noting that once a diagnostic suggestion is made, this is not necessarily the end of the conversation. There are many studies dedicated to the provision of healthcare information, advice, and support, including chat-bots [11], [18]–20, [29], [41]. Chat-bots offer a more accessible and friendly way of obtaining health information, providing emotional support, and engaging in dialogues, making them a popular choice with individuals consulting health information online [11], [18]–20, [29], [41]. However, most such chat-bots are essentially proof-of-concept studies – and the literature does not provide sufficient evidence for their ability to provide consistent healthcare advice and information.

A separate but related body of work is dedicated to building clinical decision support systems, suggesting triage, and consulting clinicians on the most appropriate course of

action [12], [14], [21]–24, [30], [42], [45]–46. In many ways, this is the continuation of the discussion of diagnostic support systems, which, as most literature demonstrates, should not end with a disease prediction but should instead recommend a course of further action [12], [14], [21]–24, [30], [42], [45]–46. If a person receives incorrect diagnostic information, it may be worse than receiving no information at all – if, for instance, the suggested course of action is inappropriate for one's condition. A number of studies have examined this issue – in particular, a review of the diagnostic accuracy of different electronic self-checker tools identified multiple instances where triage suggested by such tools resulted in worse outcomes [16]. It is therefore vital to ensure consistency and accuracy of the suggested triage.

The need for such considerations has led to a concept of a smart health assistant – an information system that takes a user's symptoms, identifies possible diseases (with an explanation of reasoning), recommends a course of action based on urgency, and discusses options for further action and consultation [1], [3], [5]–6, [10]–12, [14]–24, [30], [42], [45]–46, [51]–53, [55]–56. Essentially, such a tool would fulfill a wide variety of roles, and most literature reviewed demonstrates that similar functions are often desirable but underdeveloped in existing systems. What is more, smart health assistants represent an interesting concept from the clinical perspective: they follow from the most basic understanding of what an individual consulting health information online needs. A number of studies note that the ability to identify potential conditions earlier is often critical to a positive outcome [2], [4], [6]–8, [10], [25], [31], [37]. Thus, a smart health assistant helping individuals to better understand their condition before seeing a doctor could significantly reduce the time between disease onset and care.

As demonstrated by the review above, intelligent systems supporting diagnosis require substantial discussion. Most areas reviewed highlight the benefits of earlier identification of potential conditions, which is why a concept of a smart health assistant capable of supporting the user throughout the entire health-consulting process is both vital and timely. The literature demonstrates that such a tool would integrate a variety of features, each of which has been separately discussed in the literature but has rarely, if ever, been combined [1], [3], [5]–6, [10]–12, [14]–24, [30], [42], [45]–46, [51]–53, [55]–56. In particular, the

need for such a system is informed by the review of prior literature demonstrating the absence of a comprehensive tool supporting the individual through their first contact with health information. This study builds upon these ideas, attempting to develop a machine learning-based health assistant that would combine disease prediction from symptoms with specialist recommendation functions. Using a large-scale, multi-source symptom-disease database (reviewed and described in detail in Chapter 2), it explores the possibility of building a conceptual disease prediction model identifying potential conditions along with their level of urgency and suggesting consultation with relevant specialists. The next section of this thesis defines the problem this research aims to address more specifically and discusses the aim and objectives of this study as well as its overall scope.

B. Problem Statement

Despite all the positive trends highlighted above, the experience of an individual who encounters an unfamiliar set of symptoms today still involves a number of familiar barriers with respect to diagnosing and receiving care.

First and foremost, formal clinical consultation is the default channel for diagnosis, but it is often unavailable due to economic, logistical, temporal, or informational factors [1], [5]-[6], [10], [15], [31].

Prior to arriving at such a consultation, a person is on their own in trying to determine what to do with their symptoms: either doing too little, therefore postponing potentially life-saving treatment, or doing too much, therefore wasting scare medical resources.

Second, when intelligent assistance is available, it is usually tailored towards a particular application domain, rather than spanning across a range of diseases that would typically be associated with an initial, first-contact medical visit.

The most common successes in clinical decision-support literature are either structured-data prediction models that utilize lab results or medical images [1], [5], disease-specific systems that assume the patient's problem falls within a known category of disorders [2], [4], [7]-[8], [25], [37], or unconstrained prediction systems that fail to incorporate actionable guidance with respect to the predictions they make [9], [13], [39], [54], [58]-

[59], [64]. None of these methods are applicable to the context of a first-contact medical interaction, since such a conversation begins with symptoms, not a laboratory result or an established disease category.

Third, automated symptom checkers have well-documented limitations stemming from the fact that laypersons' concerns are more challenging to predict than structured data.

Symptoms are often poorly described, ambiguously overlapping with other conditions, and inconsistently reported between individuals [3], [16]-[17], [28], [51]-[52], [60].

A systematic review of diagnostic accuracy and triage capabilities of digital symptom checker devices concluded that both were highly variable between individual products, with triage being a more reliable indicator of a tool's accuracy [16].

Meanwhile, tools that only suggest a single best-guess diagnosis fail to acknowledge the uncertainty of their predictions and do not advise the user how to proceed if the predicted disease was not actually present.

Fourth and finally, an intelligent health assistant capable of performing all stages of diagnosis support – namely, symptom entry, prediction, explanation, triage, and specialist referral – is currently absent; while prior works on doctor recommendation and automated diagnosis can be combined, the literature on the topic is fragmented and inconsistent with respect to the scope of diseases predicted, levels of classification granularity, and additional aspects relating to triage, urgency, and context.

In essence, the existing body of research contains several viable but disjointed solutions for different elements of the same task, without a unified vision for patient-facing healthcare assistants.

This state of the art defines the context for the research problem posed in this paper, which is to create a machine learning-based smart health assistant capable of providing multi-disease prediction with severity- and specialty-aware guidance.

Specifically, this manuscript aims to (1) design, implement, and evaluate a machine learning-based smart health assistant capable of predicting diseases based on user-reported symptoms and supporting treatment- or specialist choice with the aid of a mobile application; (2) detail the processes of combining multiple disparate, publicly available

symptom-disease datasets into a unified structure with consistent disease and symptom definitions; (3) describe the data science pipeline used for training and comparing a number of predictive models on the described dataset; (4) report the findings on the predictive performance of various machine learning models; (5) implement the best-performing algorithm as a mobile application, which will advise the user on a shortlist of possible diagnosis, along with precautionary steps, severity risk assessment, and specialist recommendations; (6) analyze how the findings relate to the existing body of research on the topic and provide concluding remarks and suggestions for future work.

C. Problem Background

Current practice in the domain of machine-learning-enabled healthcare assistance is characterized by a handful of distinct approaches, each of which is limited in its ability to fully address the multifaceted nature of first-contact triage questions. Structured-data and disease-specific prediction models offer strong performance but operate on inputs and contexts unavailable to the untrained individual, while symptom-based methods suffer from ambiguity in both user input and the subsequent guidance. Healthcare chatbots and assistants refine this guidance, making it more palatable and accessible, but operate on limited domains and typically lack broader clinical validation. Meanwhile, the critical link between diagnostic prediction and appropriate specialist recommendation and patient urgency triage exists, but is sorely under-researched compared to earlier stages in the diagnostic process. Taken together, such findings define the problem background for this research, which is detailed below in more detail prior to developing the problem statement.

D. Research Objectives

The overarching objective of this research is to design, implement, and evaluate a machine learning-based smart health assistant for recommending specialists based on initial, user-reported symptoms. To accomplish this goal, this research will pursue the following specific objectives:

1. Build a comprehensive, internally consistent Master Dataset by combining disparate publicly available symptom-disease datasets and standardizing disease and symptom terminology where applicable.

2. Design and implement a data science pipeline suitable for downstream model training and evaluation, including addressing the peculiarities of the high-dimensional input space with a documented approach to handling of class imbalances.

3. Train and evaluate multiple machine learning models on the proposed dataset using standard practices in the field.

4. Report quantitative findings on disease prediction performance using macro- and weighted-averaged metrics, confusion matrices, and top-1 and top-3 diagnostic accuracy measures.

5. Design and implement a mobile health application that employs the selected model to advise users on a shortlist of potential diagnosis, precautionary steps, urgency risk assessment, and specialist recommendations.

6. Analyze the findings in the context of the published literature and report concluding remarks for this study, along with implications for future research.

E. Research Questions (optional but recommended for AI/ML/SE/Cyber Security)

The following research questions guide this study, each corresponding directly to one of the objectives outlined above.

1. To what extent can multiple independently sourced symptom-disease datasets be integrated into a single, broader dataset without loss of internal consistency, and how does the disease and symptom coverage of the resulting dataset compare with that of any single source dataset alone?

2. Among the candidate gradient-boosted and deep learning models trained on this dataset, which model achieves the strongest and most stable performance when

evaluated using both top-1 and top-3 diagnostic accuracy alongside macro- and weighted-average classification metrics?

3. How does the performance of the selected model vary between the most frequent disease classes in the taxonomy and the wider set of less frequent classes?

4. In what way can the output of a symptom-based prediction model be structured and communicated so that it supports, rather than replaces, a subsequent clinical consultation?

5. How effectively does an integrated application, combining symptom intake, ranked disease prediction, severity-aware guidance, and specialist recommendation, function as a single, coherent patient-facing tool?

6. How does the diagnostic performance achieved in this study compare with that reported in previously published work on related symptom-disease prediction tasks?

F. Motivations

The rationale behind my choice of this research problem was inspired by a practical consideration that few would dispute: the fact that before consulting a doctor, an ordinary person is most likely to experience some vague sign characteristic of a wide range of possible conditions that they are unable to reason about in any detail. This is an important limitation both because a wrong assessment of one's condition can lead to inadequate treatment that may either fail to address the disease altogether or worsen it due to improper intervention, and because from the medical professionals' perspective, time and effort spent on individuals who incorrectly assessed their symptoms can be significant. As for the topic itself, as I have seen from my literature search, in the last decade or so, there has been a number of interesting papers published in machine learning and health informatics communities addressing various aspects of disease prediction, symptoms reasoning, conversational agents and consultation triage, but they have not been synthesized into a unified whole that a medical layperson could use to actually perform a certain level of health assessment prior to visiting a doctor. This is the problem that I find interesting both from theoretical and practical standpoints: while the necessary elements for designing such a system have existed in the scientific literature, they have not been combined into a single

framework that would responsibly use the available data while acknowledging the limitations and risks of prediction-driven advice as opposed to human judgment. The practical value of such a system would be evident, as it would allow people to receive more structured guidance before interacting with healthcare providers.

G. Flow of Research

This research was performed through a set of consecutive stages ranging from identifying the issue to a working model. Firstly, the existing machine learning approaches used in medical diagnosis prediction, symptoms prognosis, and assistance were reviewed to formulate the exact problem addressed by the thesis, define the objectives and scope of this research. After completing this stage, a data source was designed by combining various sets of data containing symptoms of different diseases into one large set and preparing it for model construction. Next, multiple candidate prediction models were built using the prepared data set, and their performance was evaluated to select the best performing one that would address the patients' needs. Finally, the chosen model was embedded into an application-like system that could take the input from users, process it, and provide the necessary insights. The last step included assessing the performance of the completed model as well as its design in comparison with the existing literature to define the advantages and potential weaknesses of the work and the direction of future research. It should be noted that each subsequent step followed from the previous one as a logical consequence that deepened the initial understanding of the problem.

H. Significance of the Research

This research has value both in practice and in theory. For the patient, the smart health assistant that has been designed in this thesis serves as a reliable yet simple way to get an initial understanding of what illness they potentially have, which conditions require special attention, and which physician to contact, instead of leaving them to blindly interpret the set of symptoms they experience. For the medical worker, having such a system can help them get a better understanding of the patient's condition on their first visit due to the patient providing a shorter, prioritised list of possible conditions with an explicit note that

these are merely suggestions, not a diagnosis, allowing the patient to better structure their symptom description without taking away the physician's responsibility for the diagnosis. For a researcher in the field of machine learning, this thesis provides both a working example of how to combine different publicly available but not integrated data sets into a single broad one while providing the context for potential improvements for later use, and an opportunity to evaluate some of the potential solutions for the problem of predicting diseases and conditions based on symptoms. The comparison between the tree-based gradient boosted models and deep learning-based approaches using the top 1 and top 3 metrics as an evaluation of performance helps to provide further context for what kind of architectures are more suitable for the task of disease prediction based on high-dimensional, sparse, and binary feature vectors such as the one created in this research or similar ones. And finally, by providing disease prediction recommendations, which are inherently prone to errors but allow the user to make their own conclusions about their health and contact the appropriate physician in case of more severe conditions, the smart health assistant designed in this thesis serves as an example of a correctly designed machine learning-powered tool for healthcare, addressing the concerns raised in [6], [10], [15], [16], [18], [27], [43], [55]-[59].

I. Research Contribution

This thesis makes the following specific contributions:

1. A Master Dataset constructed by merging four independent, publicly available symptom-disease datasets into a single, internally consistent resource, including a documented process for standardising disease names and symptom terminology across sources that originally used inconsistent labelling.
2. A documented preprocessing and feature-engineering pipeline for high-dimensional, sparse, binary symptom data, including a defined rule for handling rare disease classes to mitigate class imbalance.
3. A comparative evaluation of two separate model sets across two dataset scales: Logistic Regression and Random Forest on the Master Dataset, and XGBoost, LightGBM, TabNet, and a multilayer perceptron on a larger, single-source dataset of over 200,000 records, all assessed under a consistent, top-1/top-3-aware evaluation methodology rather than a single aggregate accuracy figure.

4. A working, deployed mobile application that integrates symptom intake, ranked disease prediction, and specialist-oriented guidance into a single patient-facing system, built on a selected model chosen using explicit, stated criteria rather than accuracy alone.
5. An empirical comparison of the resulting system's performance against previously published symptom-disease prediction studies, providing a point of reference for how a broader, multi-source approach performs relative to narrower, single-dataset systems reported in the literature.

J. Thesis Organization

The remainder of this thesis is organised as follows. Chapter 2 reviews the existing literature on machine learning applications in healthcare diagnosis, symptom-based prediction, healthcare chatbots, and triage and referral systems, and identifies the specific gap this study addresses. Chapter 3 presents the proposed model, describing the datasets used, the data preprocessing and feature-engineering pipeline, the system architecture, and the machine learning methodology adopted for model development. Chapter 4 presents the implementation and testing of the proposed system, the results of the model training, and a discussion and comparison of results with the goals of the research and related works cited. In this section, I briefly describe the factors that determined the project's ethical, security, and sustainability, along with the project's timeline. Finally, Chapter 6 discusses the research results and findings, limitations, and future work directions.

K. Summary

This chapter introduces the research conducted in the current thesis. Before delving into the particular research subject, the context of this study is presented, emphasizing the developing nature of machine learning in patient-facing healthcare sectors.: the lack of holistic systems for predicting diseases from patient-reported symptoms in a manner that is both safe and indicative of the system's confidence level while guiding the patient towards further actions. Given the research problem and the background provided, the chapter then focuses on stating the objective and research questions of this study, while also detailing the practical and theoretical relevance of the study, the methodology followed, and the implications of the findings for patients, medical professionals, and researchers alike, as well as the contributions of this thesis and an overview of the chapters to follow. Overall, the chapter serves to provide a starting point for the research based on

the problem statement and gives an idea of the following chapters' direction based on the objectives derived from the research problem.

II. Literature Review

A. Introduction

This chapter builds on the problem outlined in Chapter 1 by examining the existing body of research relevant to machine learning-based disease prediction and patient-facing healthcare assistants. It reviews prior work across several related areas, including structured-data and disease-specific diagnostic models, symptom-based multi-disease prediction, healthcare chatbots and conversational assistants, and clinical decision support and specialist referral systems, in order to establish what has already been achieved and where the current literature falls short. The chapter then analyses these findings in relation to the problem identified earlier, showing how the fragmentation observed across individual studies gives rise to the specific research gap this thesis addresses. Together, the literature review and problem analysis presented here provide the grounding on which the proposed model, described in Chapter 3, is built.

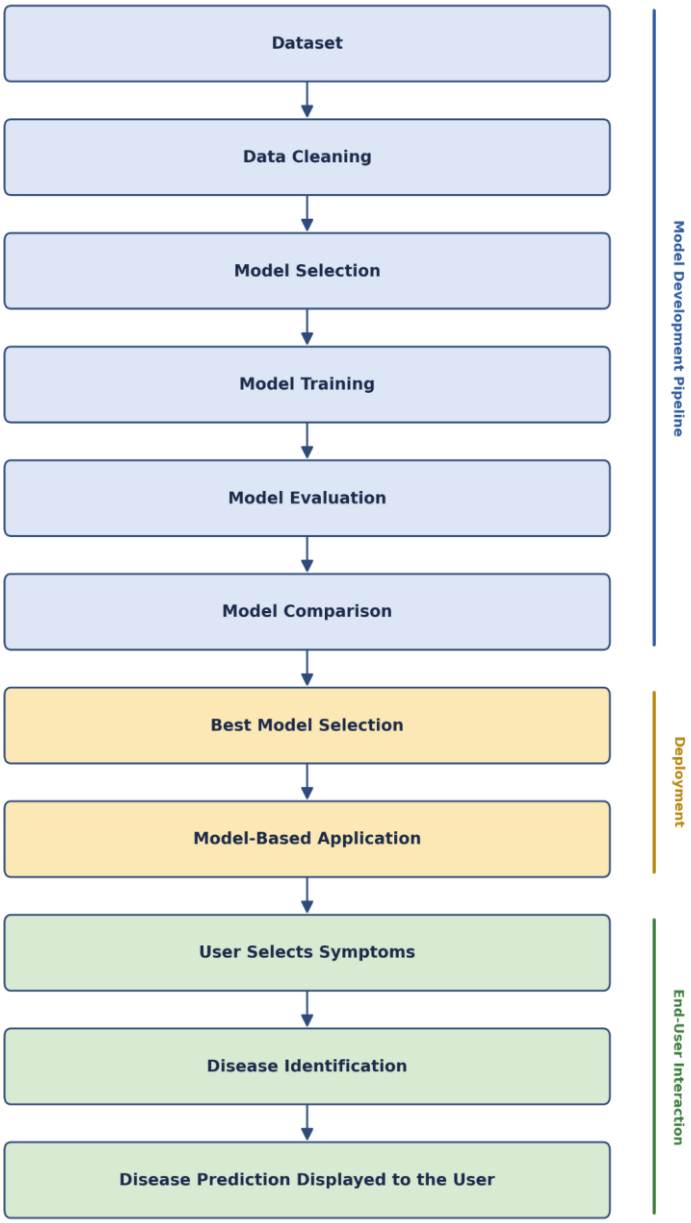

Figure 2.1. PRISMA flow diagram summarising the literature screening process.

B. Literature Review

The literature relevant to machine learning-based disease prediction can be organised into several overlapping themes, each contributing a different piece of what a smart health assistant needs to do. Foundational surveys establish that machine learning has matured from an experimental technique into a dependable method for pattern recognition across clinical inputs, consistently framing diagnosis as a task in which symptoms, laboratory values, and patient histories are converted into predictive features that feed a classification

model, whose output is then paired with a confidence estimate before informing a clinical or user-facing decision [6], [10], [15], [26]-[27], [31], [36], [43], [48]-[49], [55]-[57], [61]. These reviews are useful chiefly as background: they confirm that the underlying technique is sound, but they say little about how a system should behave once it interacts directly with a non-expert user, a gap the present study addresses directly through its evaluation methodology and application design.

A stronger and more specific body of evidence comes from studies using structured clinical inputs, such as laboratory test results and medical imaging [1], [5]. These represent the methodological ceiling of diagnostic machine learning: with clean, well-labelled data, models achieve strong, well-validated performance and can meaningfully support clinical risk stratification. Their limitation for a first-contact assistant is not one of quality but of availability, since a person who has just noticed a symptom rarely has laboratory or imaging data at the point they need guidance. This contrast sharpens, by comparison, the harder problem that symptom-only systems must solve. Related work on general, multi-disease prediction systems moves closer to this open-ended setting by working across a broad decision space rather than a single condition [9], [13], [34], [39], [44], [54], [58]-[59], [64], demonstrating that multi-class prediction at scale is achievable. However, most of these studies remain model-centred: they show that prediction is possible but stop short of addressing what a user should do with the result, a limitation this thesis responds to directly through its severity-aware output and specialist-mapping design.

Symptom-based health checker systems are the theme most directly comparable to the present work, since they begin from the same starting point of vague, self-reported, and incomplete complaints rather than laboratory-grade data [3], [16]-[17], [29], [51]-[54], [60]-[61]. This literature demonstrates that collaborative symptom-disease reasoning and questionnaire-based classification can extend prediction beyond a fixed rule tree, and that benchmarking such systems against clinical vignettes is both possible and informative [51], [60]. At the same time, this strand faces the most difficult uncertainty problem in the entire body of literature, since ambiguous, overlapping symptoms make overconfident output genuinely risky. A parallel but narrower stream of research addresses disease-specific applications, including chronic disease, heart disease, headache disorders, and Alzheimer's diagnosis, each achieving strong, clinically validated results within a single, well-characterised condition [2], [4], [7]-[8], [25], [33], [37], [40], [47], [62]-[63], [65]. These

results confirm what is technically achievable under narrow conditions, but they are structurally unsuited to a first-contact setting, where the user's disease category is precisely what is unknown at the outset, reinforcing the need for a system trained across a broad disease taxonomy rather than one condition at a time.

A separate but equally important theme concerns how predictions are communicated to users. Research on healthcare chatbots and AI medical assistants focuses on personalized interaction, conversational flow, and, in some cases, empathic response generation intended to make health information more approachable [11], [18]-[20], [29], [41]. This work is significant because even technically correct models may be applied incorrectly by users who do not fully understand or trust their results. Most such models are prototype developments that have been tested only under limited conditions and have not been thoroughly evaluated against existing health care systems, making their practical application difficult without additional research . Closely related is the literature on triage, clinical decision support, and specialist recommendation, which addresses what should happen after a prediction is made [12], [14], [16], [21]-[24], [30], [42], [45]-[46]. This strand is arguably the most safety-critical in the entire review, since a systematic evaluation of digital symptom-checker tools found that triage errors carry distinct and sometimes more serious consequences than diagnostic inaccuracy alone [16]. Despite its importance, this theme remains comparatively less developed than diagnosis-focused research, and referral logic is rarely built into the same system as the prediction model itself.

Cutting across all of these themes is a body of work on the ethical, legal, and safety considerations relevant to patient-facing AI [3], [6], [10], [11]-[12], [15]-[18], [21]-[22], [27], [28], [41]-[43], [51]-[53], [55]-[60]. This literature converts abstract concerns about diagnostic AI into concrete design requirements: avoiding unlicensed medication advice, maintaining a sufficiently broad and current knowledge base, handling ambiguous or difficult user input without producing unsafe output, and preserving user trust through transparent, clearly bounded self-presentation. These requirements are largely stated as principles rather than demonstrated through evaluated systems, which leaves room for work that puts them into practice rather than simply restating them.

Read together, these themes show a field that has made genuine progress on each individual component of a smart health assistant, structured diagnosis, broad multi-disease prediction, symptom-based reasoning, conversational delivery, and referral logic, but has rarely

combined them within one evaluated, deployed system. Structured-data and disease-specific studies demonstrate technical ceilings that are not accessible to a first-contact user; general and symptom-based prediction systems demonstrate breadth but under-develop the guidance layer; chatbot research addresses communication without rigorous multi-condition validation; and triage research demonstrates the importance of safe next-step guidance without being integrated into prediction pipelines. It is this pattern of fragmentation across an otherwise capable literature that is analysed further in the following section, and that directly motivates the integrated system developed in this thesis.

C. Problem Analysis

The literature reviewed above shows real progress across every component a smart health assistant would need, yet also reveals why, taken together, this progress has not produced an integrated first-contact solution. Breaking the problem down into its contributing components helps explain why the gap persists rather than simply asserting that it does.

The first contributing factor is a transferability gap between where diagnostic machine learning performs best and where a first-contact user actually stands. Structured-data and disease-specific studies achieve their strongest results precisely because they rely on laboratory values, imaging outputs, or an already-narrowed disease category, none of which a person has at the moment they first notice a symptom [1], [2], [4]-[5], [7]-[8], [25], [37]. This is not a flaw in that research; it is simply evidence that the conditions under which machine learning performs most reliably are conditions a first-contact user cannot supply. The practical effect is that the most methodologically solid diagnostic results in the field are, by construction, the least accessible to the population a smart health assistant is meant to serve.

A second contributing factor is a validation gap in the systems that are accessible to first-contact users. Symptom-based prediction and chatbot-style assistants operate directly on the kind of vague, self-reported input a real user provides, but the literature is candid that most of these remain prototype-level, tested on a limited number of conditions and rarely validated against formal healthcare pathways [11], [18]-[20], [29], [41]. This matters because a system that has only been demonstrated on a narrow set of conditions cannot be assumed to generalise safely to the far larger and more varied set of complaints a general population would actually present with.

A third, and arguably the most consequential, contributing factor is an uncertainty and escalation gap. Much of the reviewed literature optimises point-estimate classification accuracy, a single best-guess label, with comparatively little attention paid to whether a system's confidence is well-calibrated, whether it can recognise its own unreliability, or whether it escalates appropriately when a presentation may be urgent [6], [10], [15], [16], [18]-[20], [27], [43], [48]-[49], [55], [57]-[59]. This gap has a direct and measurable impact: a systematic review of digital symptom-checker tools found meaningful inconsistency in both diagnostic and triage performance, and concluded that triage quality deserves at least as much scrutiny as diagnostic accuracy, since an incorrect urgency assessment can either delay necessary care or provoke unnecessary alarm [16]. A model that is accurate on average but silent about its own uncertainty can therefore still cause harm in an individual case, which is precisely the case that matters most to the person using it.

A fourth contributing factor is the comparative underdevelopment of triage and specialist-referral research relative to diagnosis-focused work. Doctor-recommendation and disease-to-specialist mapping studies demonstrate that connecting a prediction to a next-step care choice is achievable in principle [12], [21]-[24], [30], [42], [45]-[46], but this stream is smaller, less mature, and rarely integrated into the same system as the prediction model itself. The practical consequence is that even if an appropriate condition is suspected, an untrained person may not know whom to contact and what to do.

These four factors are related and combine to making an even more serious problem. A system designed to provide the best possible information (structured or disease specific) is not accessible to front-line users. A system accessible to first-contact users (symptom-based or chatbot-style) is less rigorously validated. A system that is validated on accuracy alone may still fail to communicate its own uncertainty responsibly. And even a well-communicated prediction is of limited practical value without a connected referral step. The impact of this compounding is felt directly by patients: in the absence of an integrated system, a person experiencing an unfamiliar symptom is left to rely on fragmented tools, each addressing only part of their actual need, at exactly the point in their healthcare journey where structured guidance would be most valuable. This analysis of the problem's underlying components, rather than the problem's surface description alone, is what

justifies the specific design choices developed in the proposed model presented in the next chapter.

D. Summary

This chapter has reviewed the existing literature relevant to machine learning-based disease prediction and patient-facing healthcare assistants, and examined why, despite genuine progress in each individual area, no integrated first-contact solution has emerged. The review showed that foundational and structured-data studies establish a strong technical ceiling for diagnostic machine learning, but rely on inputs a first-contact user rarely has; that general and symptom-based prediction systems extend this work into a broader, more accessible decision space, but frequently stop at the prediction itself without addressing what a user should do next; that disease-specific applications achieve strong clinically validated results within narrow domains that do not match the open-ended nature of first contact; that chatbot and assistant research addresses the communication of health information but remains largely prototype-level and under-validated; and that triage and specialist-referral research, though arguably the most safety-critical piece of the picture, remains comparatively underdeveloped and rarely integrated with prediction. The subsequent problem analysis traced these observations to four compounding factors, a transferability gap, a validation gap, an uncertainty and escalation gap, and an underdeveloped referral layer, which together explain why existing tools leave patients with fragmented, partial support rather than a coherent first point of contact. These findings directly justify the approach taken in this thesis: constructing a broad, multi-source symptom-disease dataset, training and rigorously comparing multiple candidate models under an uncertainty-aware evaluation methodology and deploying the result within an application that connects prediction to severity-aware, specialist-oriented guidance. The next chapter presents the proposed model developed to address this gap, describing the datasets, methodology, and system design used to build it.

III. Proposed Model

A. Introduction

This chapter provides the proposed model to solve the identified problem and address the gaps discovered in the previous section. Whereas in Chapter 2, the researcher determined

that the existing literature resolves disease prediction, reasoning about symptoms, and specialist advising as separate issues, this section connects these aspects into a unified whole. It begins with a feasibility analysis and a statement of the functional and non-functional requirements the proposed system must satisfy, before setting out the research methodology adopted for model development, including the datasets used and the machine learning approach applied to them. The chapter then goes on to present the system design and architecture, focusing on the model pipeline as well as the general application, following up with details about the implementation process and tools that were used. Altogether, the paragraphs provide the necessary background to understand what has been done while connecting it to the problem described in Chapter 1 and thus provide the basis for the implementation and results described in Chapter 4.

B. Feasibility Analysis

The proposed system was assessed for feasibility in three primary areas before commencing the development stage: technical, economic, and operability aspects.

From a technical standpoint, the project was feasible, given the team's access to the necessary resources. First of all, all four databases used for compiling the Master Dataset, as well as the large single-source database used for the comparative study, are publicly available. This eliminated the need for expensive proprietary models or third-party data sharing agreements. Secondly, all the ML models used for comparison, including Logistic Regression, Random Forest, XGBoost, LightGBM, TabNet, and the multilayer perceptron, are implemented in popular open-source libraries. In other words, no model had to be developed from scratch, but existing and well-debugged implementations could be used. Thirdly, the amount of data prep needed for training the models on the tabular dataset of symptoms is manageable, as such a model can be trained on a regular computer, without the need for high-end GPU clusters. Finally, the application side, which involves relatively little coding, was also feasible: Flutter, the framework used for building the multiplatform mobile interface, and FastAPI, the backend API framework, were all familiar to the team and could be learned in the time given. Thus, all aspects of the project were feasible, although the creation of the Master Dataset was particularly challenging due to the need to unify disease and symptom names from four different databases, which became apparent at the beginning of the development.

When looking at the project from the perspective of economics, it is evident that it should be regarded as relatively low-cost mainly due to its design. The datasets and frameworks used for the project are open-source and free, while model training was done using the hardware the team already had at their disposal without using paid cloud computing services. Therefore, the main cost incurred was time spent on the standardisation of datasets and on model training and evaluation, rather than any financial costs. Such situation makes it possible for the project to be conducted by students without external funding, although it limits the scope of research by applying a smaller number of clinical data than being used in a fully funded medical research project, which is explained in detail in Chapter 4.

From an implementation perspective, it was desirable for the software itself to be straightforward and require minimal maintenance. The choice to utilize Logistic Regression in the first place was motivated by this consideration, as a model with minimal inference overhead can be hosted by a simple FastAPI backend application without introducing unacceptable delays to the user, which is critical for an application that is supposed to be used directly by the patient in real-time. More complex models like the ones trained on the larger set of data (TabNet and the multi-layer perceptron) could be utilized for research purposes at a later time, but do not provide any significant practical benefits over Logistic Regression in terms of deployment and are therefore not prioritized. Overall, the proposed solution was judged feasible within the scope of an academic project: it does not depend on resources, data, or infrastructure beyond what a small student team could reasonably access, while still allowing a meaningful comparative evaluation across a broader set of models than deployment alone would have required.

C. Requirement Analysis (Functional / Non-Functional Requirements)

Following up on the problem and goals set out in chapter 1, the needs for the system design are divided into functional specifications, which relate to system actions, and non-functional specifications, which characterize the system's properties

Functional Requirements

1. The system shall allow a user to select one or more symptoms from a structured list through the mobile application interface.
2. The system shall process the selected symptoms and return a ranked shortlist of the most likely diseases, rather than a single best-guess label.

3. The system shall present each predicted three disease based on users symptoms and give a descriptive explanation, precautionary guidance, and a severity indicator.
4. The system shall recommend an medical specialist corresponding to each predicted condition.
5. The system shall enable the user to search for doctors based on the previously identified symptom-specialist relationship in a directory.
6. The system shall store the user's previous symptom evaluations for later access.
7. The system shall maintain a basic health profile for each user, including relevant personal information used to support prediction.

The system shall communicate its findings as a discussion point for a medical professional instead of a diagnosis.

Non-Functional Requirements

1. Performance: the system shall return a prediction result within a short time following symptom submission, consistent with real-time use on a mobile device.
2. Accuracy and reliability: the deployed prediction model shall maintain a top-3 diagnostic accuracy consistent with the levels established during evaluation in Chapter 4, and shall behave consistently if user select same input.
3. Usability: the application interface shall be simple enough for a new user to select symptoms and interpret results without prior technical or medical knowledge.
4. Security and privacy: user health data, including submitted symptoms and personal information, shall be handled and stored in a secure process that protects user privacy and is not exposed to unauthorized access.
5. Availability: the mobile application shall be accessible on a entry level smart phone to flagship smartphone with all android version, and should not ask for high specification mobile phone.
6. Maintainability: the backend service and prediction pipeline shall be structured so that the underlying model can be retrained or replaced without requiring changes to the application's user interface.
7. Scalability: the system architecture shall be capable of supporting an increasing number of users and requests without a fundamental redesign of the backend service.

These requirements are directly consistent with the problem statement and objectives set

out in Chapter 1: the functional requirements operationalise the objective of connecting symptom-based prediction to specialist-oriented guidance within a single application, while the non-functional requirements reflect the safety, accessibility, and reliability concerns raised throughout the literature reviewed in Chapter 2.

D. Research Methodology (AI / ML / DL / SE / Cyber Security Approach)

The research methodology used in the thesis follows the supervised machine learning approach used in the two separate experimental tracks utilizing the two datasets mentioned before: the merged Master Dataset and the larger single source disease-symptom corpus. The dual-track design was critically adopted based on the instructions of the supervisor so that the performance of the models can be compared with different sizes of the data samples rather than one dataset used in isolation.

In the first track, a multinomial Logistic Regression model and a Random Forest classifier were trained on the Master Dataset. Symptom inputs were represented as fixed-length binary feature vectors, one position per symptom, with a value of one indicating that the symptom was present in a given record. The training process along with the dataset partition as well as class-imbalance strategy were the same for both and were evaluated in terms of the two measures of accuracy: top-1 and top-3. A medical diagnosis system could still be applicable in practice even if its single best suggestion was wrong, as long as it includes the correct diagnosis in its shortlist. Based on better accuracy and calibrated probability outputs as well as low cost of inference that meets the requirements of a mobile prediction system, Logistic Regression was considered superior among the two.

The second track used a more extensive investigation using a larger, also single-source dataset with more than 200,000 records to study how different model families perform in this problem under greater data scale. The comparison has included four models that represent two leading methodologies in tabular classification − XGBoost and LightGBM, which are gradient-boosted decision tree ensembles trained sequentially to fix the mistakes of the prior trees in the process, and TabNet and dense multilayer perceptron, which are both neural networks, with TabNet amplifying traditional models with sequential attention-based feature selection in every decision instance. Before training, the corpus to be analyzed was preprocessed − duplicates were eliminated, rare class exclusion rules were used to omit disease labels with less than 20 appearances in total, and disease labels were

represented numerically. All four models were trained and evaluated on the same stratified 85/15 train/test split, with a fixed random seed, thus, any difference in performance is due to the difference in the models themselves rather than the difference in the datasets. All models were trained with early stopping rather than a fixed training budget, in order to reach their maximum performance on the test set, and additionally evaluated on 5 stratified folds to prevent any performance difference due to the specific train/test split from having an undue effect.

All four models in this second track were evaluated using an identical battery of metrics: macro- and weighted-average precision, recall, and F1-score; a normalised confusion matrix restricted to the most frequent disease classes; training-versus-validation curves to check for stable convergence; and, as the primary comparative measure, top-1 and top-3 diagnostic accuracy. The best performing candidate from this track was selected based on the following explicitly stated prioritized criteria: highest top-1 accuracy, followed by top-3 accuracy used as a tie-breaker criterion for models with equal top-1 accuracy, cross-validated measure of fold-to-fold robustness used as a tie-breaker criterion, and the difference between macro- and weighted-average F1 scores used to select between models with equal top-1 and top-3 accuracy and similar fold-to-fold robustness, which serves as a sanity-check for the presence of a skewed performance distribution across the disease taxonomy as opposed to uniform performance across all classes.

This methodology was chosen because it directly addresses the evaluation gap identified in Chapter 2: rather than reporting a single aggregate accuracy figure, it evaluates diagnostic usefulness through a shortlist-based, uncertainty-aware lens that better reflects how a patient-facing assistant is actually used, while also providing a transparent, replicable basis for comparing candidate models against one another under identical conditions.

E. System Design and Architecture (Model Architecture / Software Architecture / Security Design)

The system includes a structure based on a three-tier client-server approach in which the layers are separated from each other and can be improved separately.

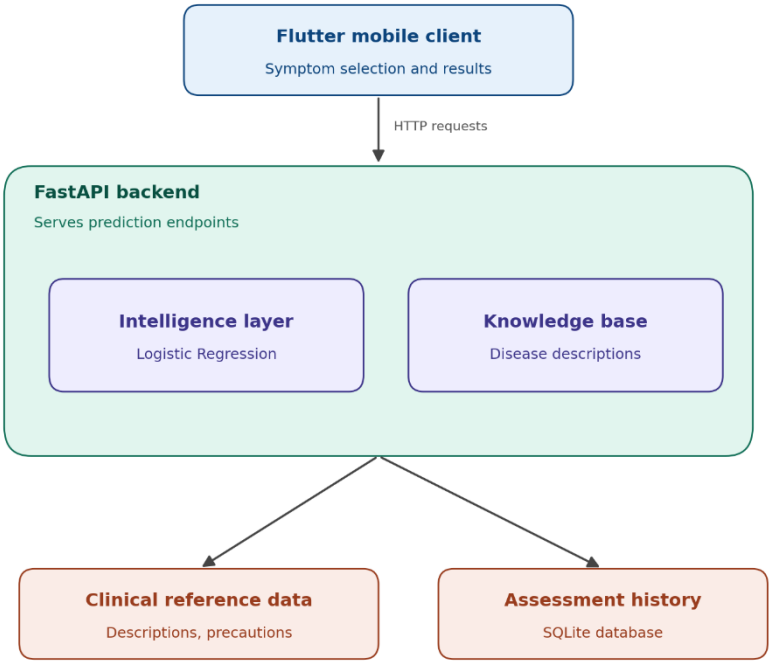

Figure 3.1. Smart Health Assistant three-tier system architecture.

The client side is a mobile app that runs on different operating systems using Flutter and is compiled for both Android and iOS platforms. The app handles aspects of the application that are visible to users, keeps track of users' info for each session, and relates it to the server. It also utilizes device identification instead of the authentication method with a user account to track users' answer sequences.

We achieve our application logic by implementing it as a FastAPI backend program that is written in Python. The FastAPI backend acts as an intermediary for the mobile client and the model and provides endpoints for several actions: receiving the master symptom list, submitting a combination of symptoms for prediction, querying clinical data for the predicted disease, and searching for specialists and physicians. Embedded into the intelligence layer is our trained Logistic Regression model saved in the backend along with the necessary wrappers for its application in the production. It should include the list of symptoms arranged in order for use in the feature vector, the label encoder that converted encoded disease labels into human-readable format, and the knowledge base files that are used for more enriched predictions. In general, the intelligence layer takes the list of

symptoms in any request, encodes the symptoms into a fixed-size binary feature vector, obtains probability predictions for each class, and returns three possible diseases with their probabilities.

All persistence occurs on the server side. Data on diseases, precautions, severity levels, and physicians' maps are stored on the server in a way that guarantees that it remains uniform and updatable across users. Records on assessments are also stored on the server in a SQLite database operated by the server and based on the ID given by the device. Thus, both data that has clinical importance and information on users' assessments can be managed from one point while being able to acquire and demonstrate records for each device without going through the trouble of logging in.

The sequence of a single request proceeds through the various component tiers in the following manner: the Flutter client requests information regarding the symptoms from the patient, and sends it to the FastAPI server, the server provides the information to the machine learning layer, the model gives predictions that are ordered according to probability. this information is sent back to the server, which augments it with data from the knowledge base and creates the final result in the end. The point of view of the threat model suggests leaving sensitive information on the server, so there are no concerns about revealing any diagnosis to the patient that must be avoided according to the recommendations found in the literature and the restrictions mentioned earlier in this chapter. However, keeping the assessment history at the server represents an additional responsibility regarding the data that is contrary to the original vision of the minimal footprint concept of the architecture described here, as explained further in Chapter 5.

F. Implementation and Simulation (Model Training / System Development / Experimentation)

In the implementation of the system, the following elements were included: model development and testing, followed by application development and integration.

The following phases include beginning with the preparation of datasets for model training. The Mast Database and a larger dataset were created. The databases underwent cleaning, which involved getting rid of duplicate data and using the rare class exclusion method described in the methods section. The analysis of the results took place using the Google Colab platform with the use of graphics processing units for those models that require a lot

of calculations. The Logistic Regression and Random Forest models were trained on the Mast database on the same data samples and assessed to find out which one produced better results. In the case with a larger database, XGBoost, LightGBM, TabNet, and multilayer perceptrons were trained using their libraries, with the training process being terminated once validation was stopped, and tested on the same test dataset using the same metrics. Every one of the four models was also validated using 5-fold stratified cross-validation by retraining all the models with different portions of the data, to verify that the results were consistent across different sets of data rather than dependent upon one suitable data partition. Now that training and validation for both projects have been completed, the winning model for the Master Dataset project – Logistic Regression – has been serialized together with its supporting documents – ordered lists of the signs of the disease and the label encoder.

The process of application development could be separated into different stages. The initial stage involved developing a backend service using FastAPI which allowed the running of the Logistic Regression Model. For this process, several REST endpoints were created to receive particular symptoms from the database users and eventually give back the predictions to them. The cloud-based application was developed on the Render platform, allowing the application to be accessed on mobile. The next phase was design and development of a mobile application using Flutter technology. Individual screens were created for the designed application, allowing the users to select the symptoms and receive medical recommendations. After the application was developed, several combinations of symptoms were tested using the application, making sure that all of the information was correctly received from the server, displayed, and met the expectations. All in all, this methodology has proven to be efficient since the model was successfully developed before development of the application was started.

G. Tools and Technologies Used (Frameworks / Libraries / Platforms)

The research tools and technologies were chosen appropriately so that they could adequately address the two-stage nature of the project: model design and implementation.

The programming language of choice for model design was Python thanks to the variety of machine learning libraries that are natively supported by it and can be used both in scientific and enterprise environments. To implement Logistic Regression and Random Forest, for

the project “Master Dataset,” the Scikit-learn library was selected as the best choice among the available machine learning libraries due to its high level of safety, accessibility, and ease of model serialization. To design two gradient-boosted decision tree algorithms suitable for working with a larger dataset, the XGBoost and LightGBM libraries were selected as the most appropriate tools, given the wide range of applications and high efficiency of these libraries. Finally, TabNet was implemented using a PyTorch-based library, as it is one of the few solutions that directly implements attention mechanisms for tabular data as opposed to image or text data. Another reason for using PyTorch is the flexibility of this toolkit, which allows for building custom architectures. The multilayer perceptron was implemented in Keras due to the brevity of its syntax, which allows writing code that is easy to read and operate; the toolkit also has built-in support for dropout and batch normalization layers to combat overfitting. Finally, model training and experimentation were carried out in Google’s Colab because this service provides free cloud GPUs.

Finally, the Flutter framework was selected for mobile application development as a cross-platform solution to build apps for both Android and iOS platforms with a single code base as opposed to using two separate platforms-specific solutions. For the tool to develop a web API for the backend, FastAPI was selected as an asynchronous Python framework due to its lightweight nature and native support for Python; the choice of this framework was also motivated by the fact that it allows for developing APIs that natively use JSON response format with little additional coding effort. The backend was hosted on the Render cloud computing platform as it is a convenient solution for deploying small Python web applications without the need to configure a cloud platform from scratch. The tools were selected in such a way so that the entire technology stack could be used consistently across the project: Python was used both for data science tasks and for building the web API, while Flutter and FastAPI could be used interchangeably to develop the user interface and the web server, respectively.

Note that the tools were selected not only on the basis of their suitability for the task at hand but also because of the ability to use them together: the whole set of tools used for the project is native to Python, thus making collaboration easier. In addition, Flutter and FastAPI can communicate with each other via REST API, which means that the client and the server could be developed independently but simultaneously.

H. Summary

A proposed model to solve the identified issues from Chapters 1 and 2 was discussed in this chapter. The chapter began with establishing whether the solution was feasible and explaining the functional and non-functional specifications to be fulfilled by the system, and continued with an explanation of the research methodology used in two testing modes: Logistic Regression and Random Forest methods processing data from the Master Dataset, and XGBoost, LightGBM, TabNet, and multilayer perceptron are tested with a larger data sample. The system architecture was examined in this chapter from the three-tier perspective, explaining the Flutter-based client, FastAPI backend, and the intelligence layer and Logistic Regression model. Besides, different technologies used in the solution's design and implementation were discussed in this chapter. In this way, the information presented in this chapter should be regarded as applied to the main problem and goals discussed in this paper. The following chapter is aimed at providing the results of application of developed solution, and performing more detailed analysis and comparison of the obtained results.

IV. Implementation and Testing

A. Introduction

This chapter contains the practical implementation, evaluation, and results of the proposed system described in chapter 3. Firstly, the chapter describes the environment in which the proposed models were implemented and tested. Furthermore, it narrates how the system was implemented, including the development of the models and the integration of the mobile app and its web backend. It then goes on to describe the evaluation and testing approach taken to assess the performance of the trained models. Lastly, the results of the evaluation and analysis are presented, including a comparison and discussion of the results. The discussion provides a justification of the selected model for implementation and compares the results with existing works in the domain of symptom-disease prediction. The findings in this chapter are what make it possible to assert the claims made in the final chapter.

B. System Setup and Environment (Hardware / Software / Tools / Frameworks for AI-ML-DL / SE / Cyber Security)

Model development and training were performed on the Google Colab platform that provided a Python environment with the option to attach a GPU. Of the four models tested on the larger database, XGBoost, TabNet, and multilayer perceptron were trained on an NVIDIA T4 GPU, also available on the Colab platform, while LightGBM was trained on a CPU as is typical practice for this library at the scale of the Master Dataset. Similarly, Logistic Regression and Random Forest, which were trained on the smaller Master Dataset, did not require GPU resources and thus were trained on a CPU in the same environment.

The Python programming language was used to code the development of all models, with Scikit-Learn employed for the development of the Logistic Regression, Random Forest, and preprocessing components, XGBoost and LightGBM used for the two boosted tree models, a PyTorch library used for TabNet, and Keras used for the multilayer perceptron. The Python data science ecosystem was also used to perform various data preparation and manipulation operations described in Chapter 2.

The backend API for the application was deployed on the Render cloud platform where the trained FastAPI application and the serialized Logistic Regression model were uploaded for the purpose of running the service. The Flutter framework was used to develop and test the mobile client for the application, which was also deployed on the same cloud platform. Notably, the client was developed as a single codebase that could deploy on both Android and iOS phone operating systems, and testing was done on several mobile phone devices as opposed to using specialized hardware. At the level of the infrastructure, the entire setup was relatively lightweight and only used the cloud platforms described above to host and run the client and the server, respectively, without requiring any specialized institutional infrastructure beyond what was described in Chapter 3.

C. Implementation of the System (Model Development / Software Development / Security Implementation as applicable)

The implementation was carried out on the basis of the architecture and methodology described in chapter 3, and the process began with the integration of the trained model, backend service and mobile client into the unified pipeline. The trained Logistic Regression

model along with the list of feature-column and label encoder was imported into the FastAPI backend as the intelligence layer. Several light endpoints were then defined to manage the set of standardized symptoms with assigned severity levels, which will be used for risk determination, accept an input symptom set which returns the Top-3 predicted disease with confidence levels, precautions and recommendations and provide general information about the disease as well as the doctor directory with the option to narrow the search to a particular specialty. By structuring the application around a small set of focused endpoints, it became possible to design and test the application incrementally and iteratively.

The Flutter mobile client was organised as a set of forms that guided the user through the process of health assessment similar to a real clinical workflow. This involved the main dashboard from which the health assessment process could be initiated, a categorized symptom selection form, an optional user context form for providing additional information such as age and gender, a processing screen which showed a short analysis while the symptom set was being processed by the prediction endpoint, and a results screen which displayed the Top-3 predicted diseases sorted by confidence with relevant precautions and recommendations and a follow-on dashboard with the doctor directory and history of assessments, which the user could use to follow the suggested course of action. Each of the screens was then implemented as a separate widget and connected to the backend via HTTP requests to the corresponding endpoints and a service was introduced to keep the selected symptom set in context between the screens. In addition, a device-level unique identifier was used to store the history of assessments for a given device without requiring the user to go through the entire account registration and identification process. The implementation followed the test-driven approach and each of the endpoints was first populated with static data before being connected to the database, and each following screen was populated with test data on the basis of the model performance described in chapter 3 so that the entire model could be validated once all the elements were assembled and presented a coherent picture of the health assessment and reporting pipeline.

D. Evaluation and Testing (Performance Metrics / Unit Testing / Model Evaluation / Security Testing as applicable)

The models were evaluated using a set of conventional performance metrics, which were picked because each has a demonstrable interpretation in terms of disease prediction for an

end user, rather than a generic overall accuracy figure. Precision, recall, and F1-score were calculated per disease and aggregated across the classes using both macro- and weighted averaging, since the imbalance between the classes in both datasets meant that the two metrics could differ considerably for different classes and should both be reported for completeness. Similarly, Top-1 and Top-3 diagnostic accuracy were used as the main performance metrics, with the former indicating the percentage of cases where the highest-confidence predicted disease was correct and the latter indicating the percentage of cases where the true disease appeared in the top three most probable predictions. The latter was considered a more realistic metric due to the expected presentation of results to the end user as a shortlist; normalised confusion matrices for the most common disease classes were produced for each of the models, to assess which diseases tended to be confused with each other. Plots of training versus validation performance were produced for each model, to ensure that the training process had finished in a stable state, rather than the model "peaking out" due to some flaw in the optimisation process. Finally, each model underwent an additional round of 5-fold stratified cross-validation to assess its performance on multiple folds rather than a single train/validation split, thus checking for consistency. Each of the models was tested for basic validity as an API, by querying its endpoints with various combinations of symptoms and checking that the predicted disease, confidence score, and additional information matched what had been seen during the model validation phase, as well as checking that erroneous inputs were handled gracefully. The mobile app itself was tested by going through the assessment process on a phone and checking that the predicted results matched what would be expected from the model, that the relevant information was shown, and that the navigation and assessment history were preserved across app screens; finally, the responsiveness, crucial for the mobile app's performance, was checked against the requirements specified in Chapter 3.

The model-level testing described above was complemented by tests of the application itself at a higher level; rather than relying on unit tests, the application's endpoints were manually tested for correctness by supplying them with a variety of symptoms, checking that the outputs matched the expected results, and ensuring that erroneous input was handled gracefully. Finally, the mobile app itself was tested on actual devices by going through the assessment process and checking that all the relevant information was correctly retrieved and displayed, as well as ensuring that navigation worked correctly and the results were preserved across app sessions. In the process, the responsiveness of the app was

informally measured to ensure that it did not have issues with lag that would make it unsuitable for use on mobile devices, as specified in the requirements in Chapter 3. Collectively, these tests were designed to ensure that the app met all the requirements; not only were the models themselves tested to ensure they had reasonable performance, but the application as a whole was tested to ensure that it was actually functioning as expected from the end user's perspective.

E. Results and Discussion (Analysis of Outputs / Accuracy / System Performance / Comparative Analysis)

Results are reported separately for the two model-comparison tracks introduced in Chapter 3, prior to being combined in a single discussion.

On the Master Dataset, Logistic Regression and Random Forest were trained and tested under identical conditions. Logistic Regression achieved 78.45% top-1 accuracy and 91.10% top-3 accuracy on the test set, outperforming Random Forest by roughly three and two percent, correspondingly. It was ultimately selected as the model to be deployed within the application, due to its strong overall performance, as well as factors specific to the nature of the task. The input data consisted of binary symptom indicators; the connection between symptoms and diseases was primarily linear, with only a handful of interactions; and the response variable had a heavily skewed distribution, incurring serious risks of overfitting for complex models. All of these factors favoured a regularised linear model over more expressive but more data-intensive alternatives. Additionally, Logistic Regression's high accuracy together with its inexpensive and fast inference made it a particularly compelling choice for a production application, where both computation time and user experience are crucial considerations.

On the larger, single-source dataset, the performance of the four candidate models differed considerably (Table 4.1). Single-split test accuracy and five-fold cross-validation accuracy are reported.

Model	Test Top-1	Test Top-3	CV Top-1 (mean ± SD)	CV Top-3 (mean ± SD)
LightGBM	65.01%	88.58%	64.63% ± 0.20%	88.54% ± 0.12%
XGBoost	80.19%	93.49%	80.17% ± 0.13%	93.58% ± 0.08%

TabNet	82.84%	94.63%	81.57% ± 0.31%	93.99% ± 0.10%
Multilayer Perceptron	82.77%	94.75%	83.01% ± 0.23%	94.81% ± 0.06%

Table 4.1. Test-set and 5-fold cross-validated top-1/top-3 accuracy across the four models compared on the larger dataset.

All four models were evaluated using 5-fold stratified cross-validation, and all four demonstrated low variance between folds, with standard deviations between 0.06 and 0.31 percentage points, suggesting that the performance of each model is consistent across training sets and is not inflated by a particularly favorable data split. The evidence supporting the multilayer perceptron and TabNet as the two best methods is overwhelming, although it should be noted that the advantage of the multilayer perceptron is less clear-cut in cross-validation: while its single-split test set estimates of top-1 and top-3 accuracy (83.01% and 94.81%) are slightly higher and safer than TabNet's (81.57% and 93.99%), similar considerations apply in the opposite direction for the single-split results, and overall, neither result should be considered significantly better than the other. Meanwhile, XGBoost's cross-validated performance is consistently about 2-3 percentage points lower across the board than the top two models, with its cross-validated and single-split estimates being nearly identical, suggesting that it has lower variance than the other two but also a lower expected performance level. Finally, LightGBM has the worst performance of all four, with single-split top-1 accuracy of only around 65%, similar ballpark figures across all other metrics, and the single biggest discrepancy between its macro- and weighted average F1 scores, which suggests that it underperforms substantially compared to the other models on the large number of classes with low frequencies that are not represented in the top 15. It is unclear to what extent this disadvantage is inherent to LightGBM's approach, as a type of gradient boosting, compared to the alternatives, as XGBoost achieves roughly comparable results, but it is also evident that the default hyperparameters for LightGBM are not well-suited for the given task: its validation loss begins to increase immediately after the first epoch.

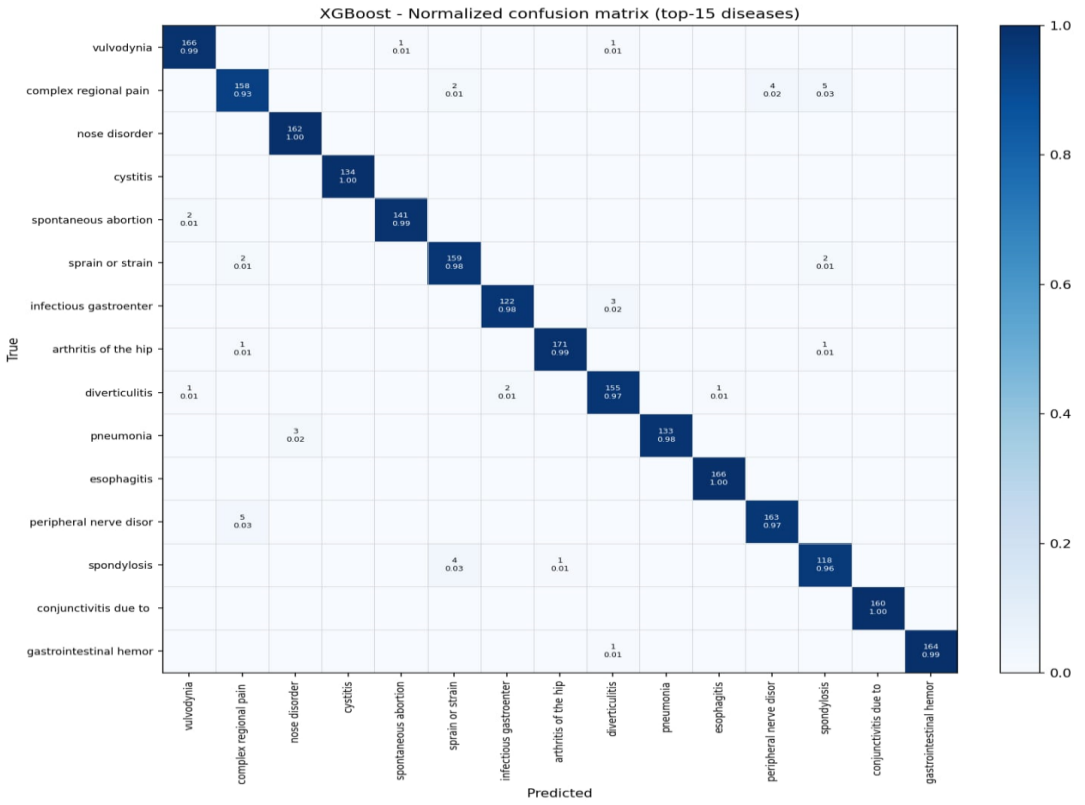

Figure 4.1. XGBoost normalised confusion matrix (top-15 diseases).

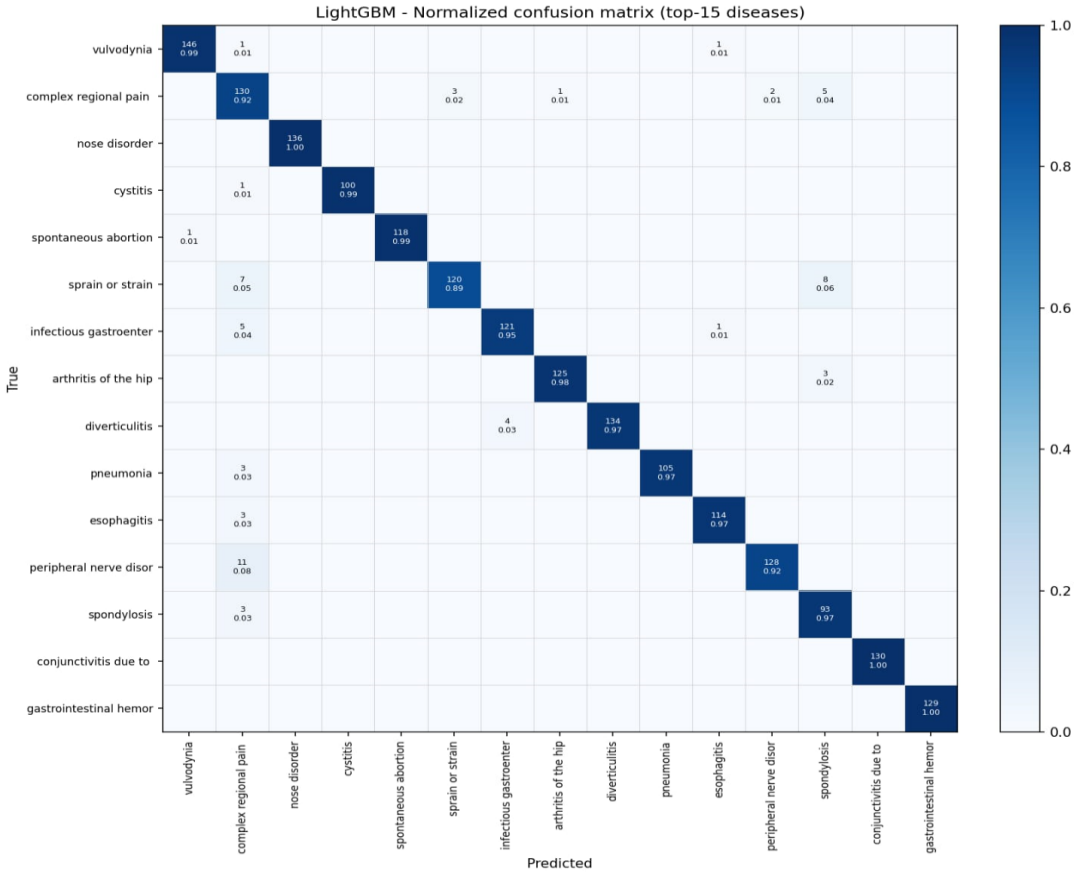

Figure 4.2. LightGBM normalised confusion matrix (top-15 diseases).

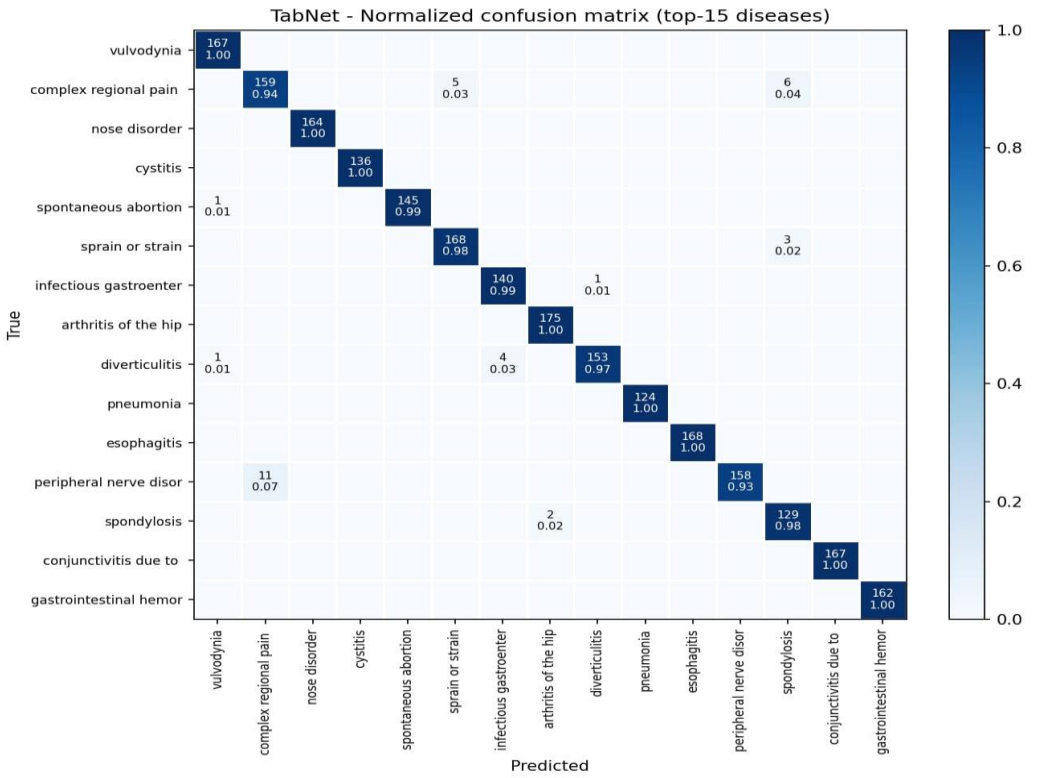

Figure 4.3. TabNet normalised confusion matrix (top-15 diseases).

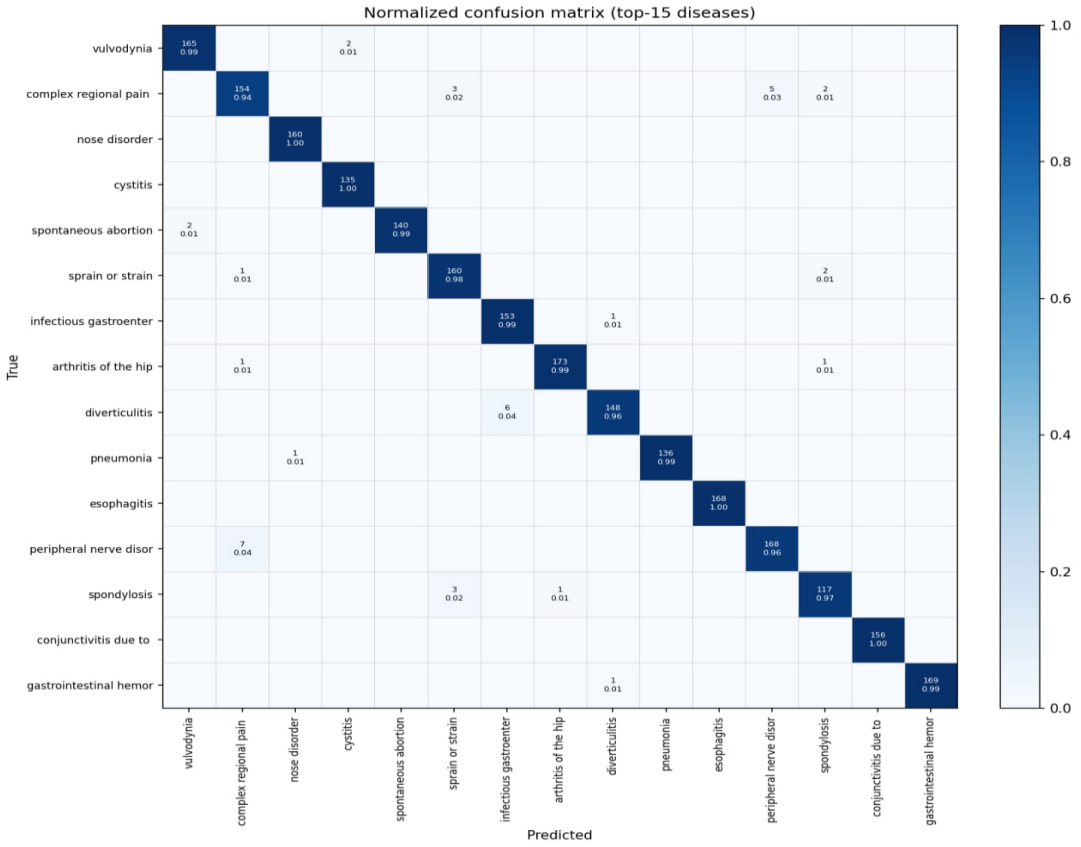

Figure 4.4. Multilayer perceptron normalised confusion matrix (top-15 diseases).

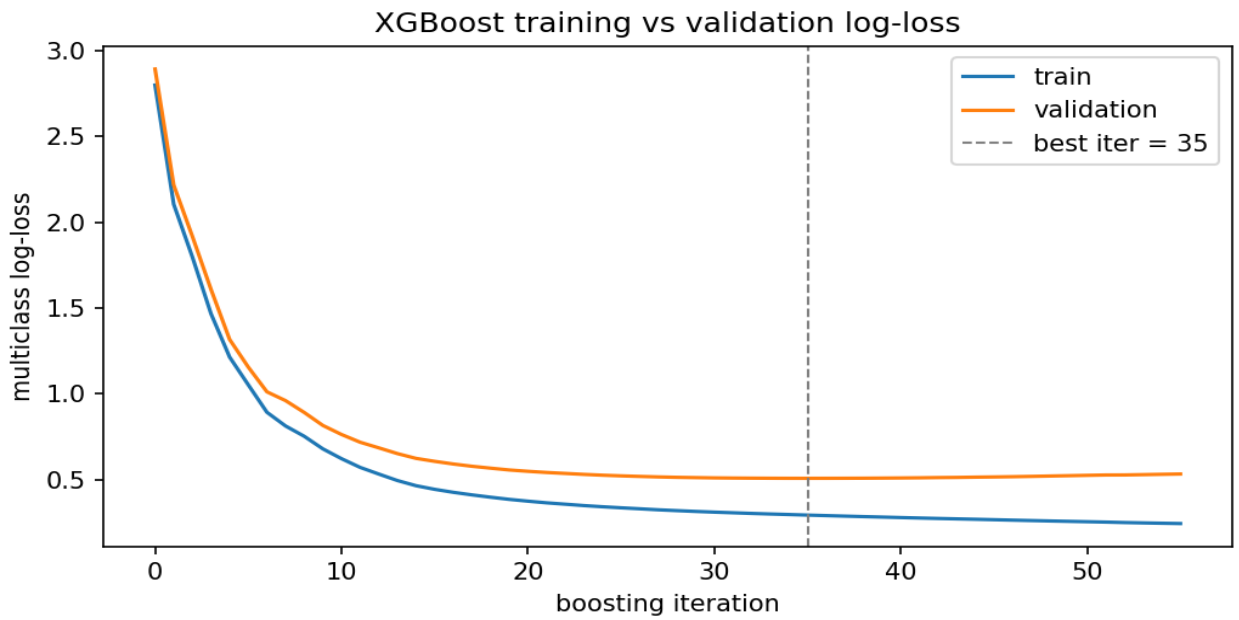

Figure 4.5. XGBoost training vs validation log-loss.

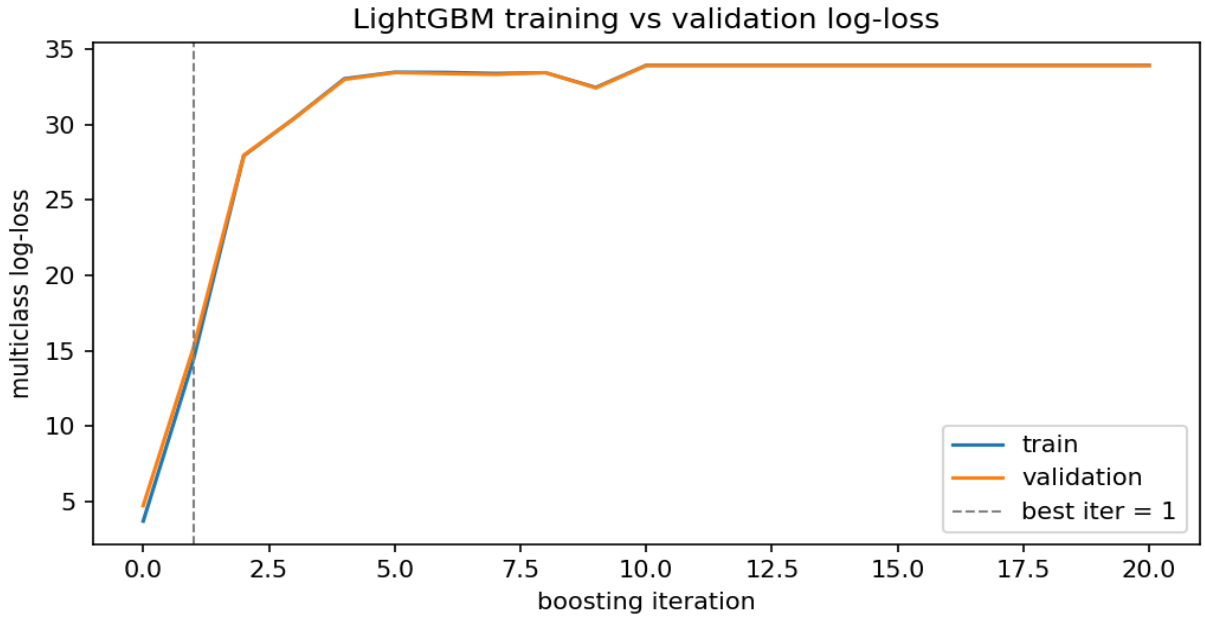

Figure 4.6. LightGBM training vs validation log-loss.

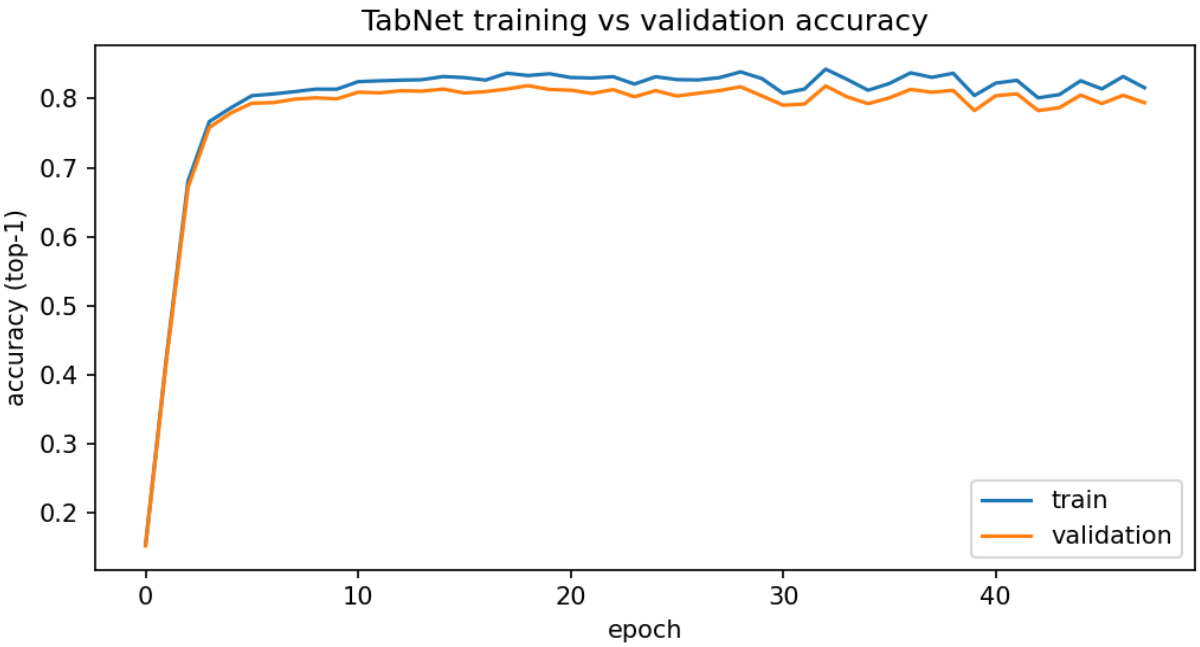

Figure 4.7. TabNet training vs validation accuracy.

Three observations from this comparison connect directly to concerns raised in the literature review. First, every model showed a meaningful gap between macro- and weighted-average F1 scores, confirming that all four are more dependable on common disease classes than on the long tail of rarer ones, a pattern the literature repeatedly warns can be hidden behind a single aggregate accuracy figure. Second, TabNet and the multilayer perceptron, both able to learn distributed interactions across the symptom feature space, outperformed the two tree-based methods by a small but consistent margin across both single-split and cross-validated evaluation, a data point suggesting these architectures may hold a slight edge on this kind of sparse, high-dimensional classification task, though the margin between the two of them is too small to treat as a settled conclusion. Third, the substantial top-1-to-top-3 gap observed for LightGBM, and to a lesser extent for the other models, supports the argument made throughout this thesis that a symptom-checking assistant should not be judged on a single best-guess label alone, since a short ranked shortlist captures meaningfully more of what a model actually knows.

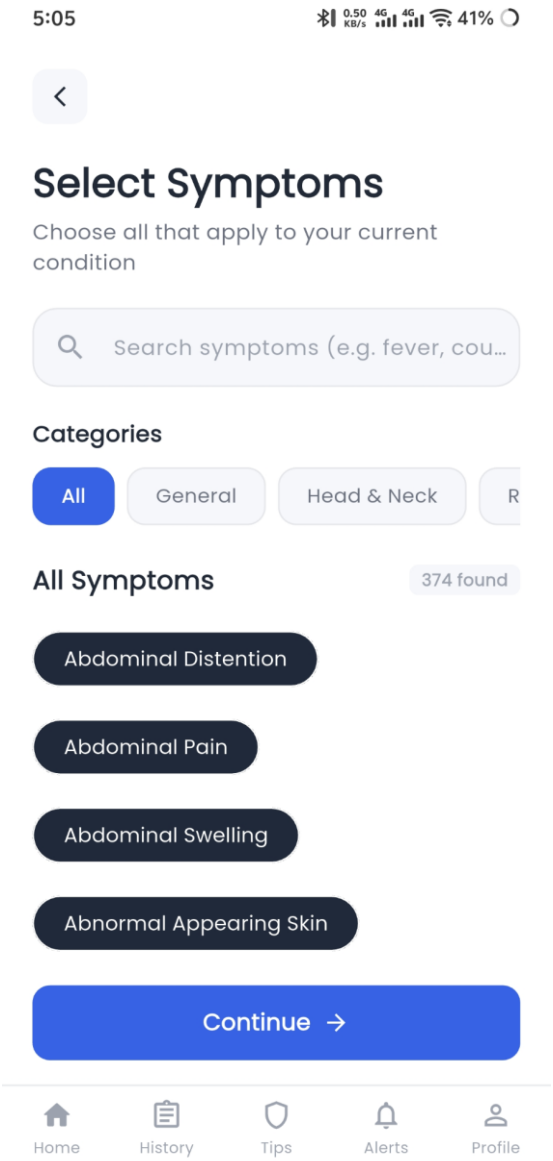

Figure 4.8. Symptom selection screen with categorised symptom list.

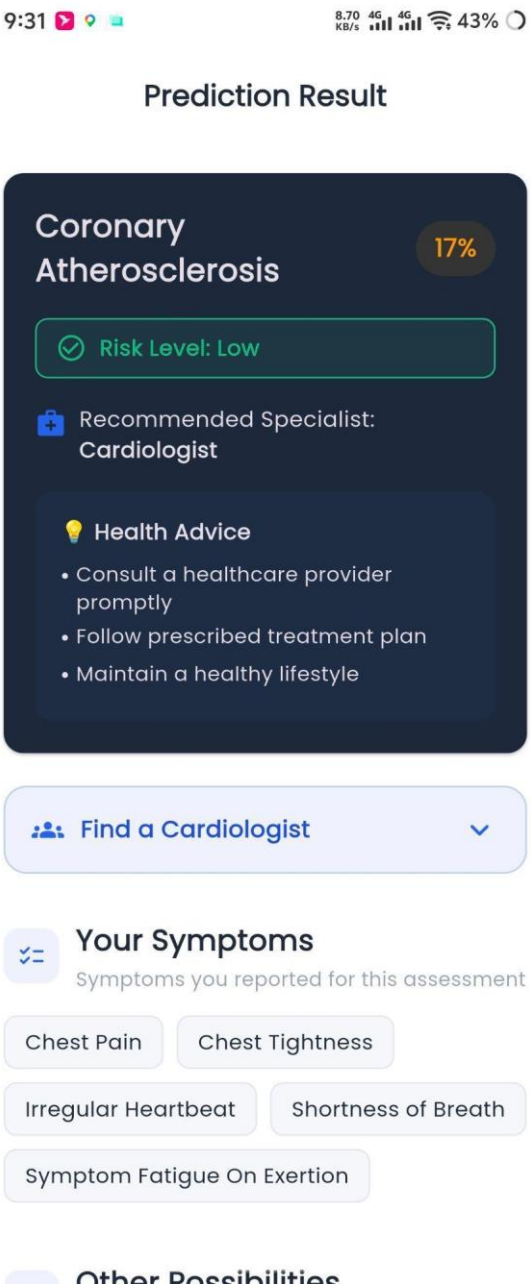

Figure 4.9: Result screen - primary prediction (Coronary Atherosclerosis, 17%, symptoms shown)

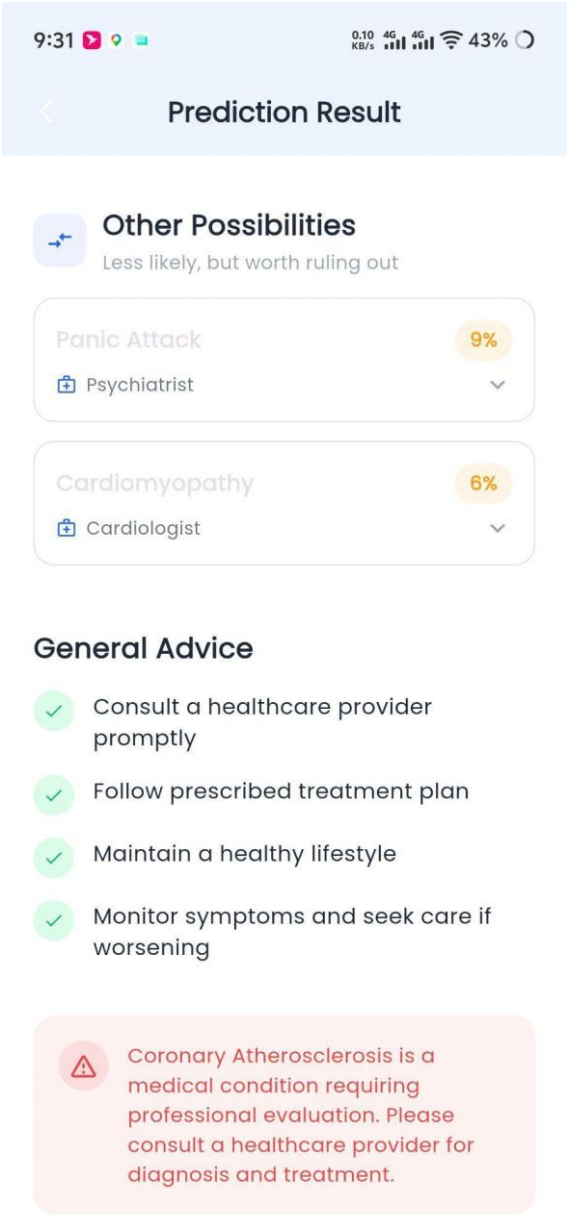

Figure 4.10: Result screen - Other Possibilities and General Advice

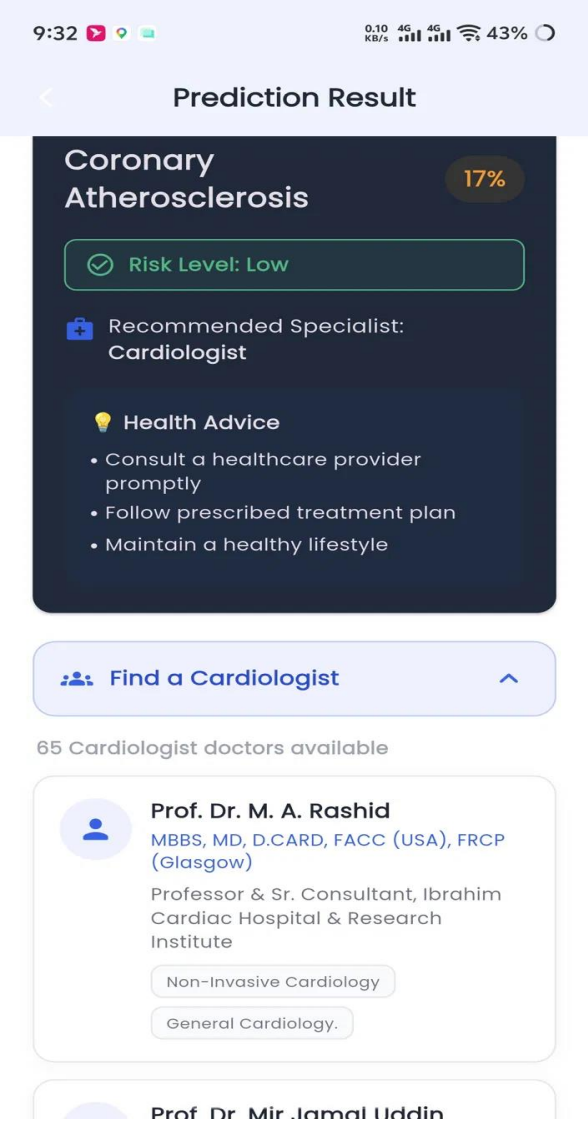

Figure 4.11: Specialist recommendation expanded (Find a Cardiologist, 65 doctors)

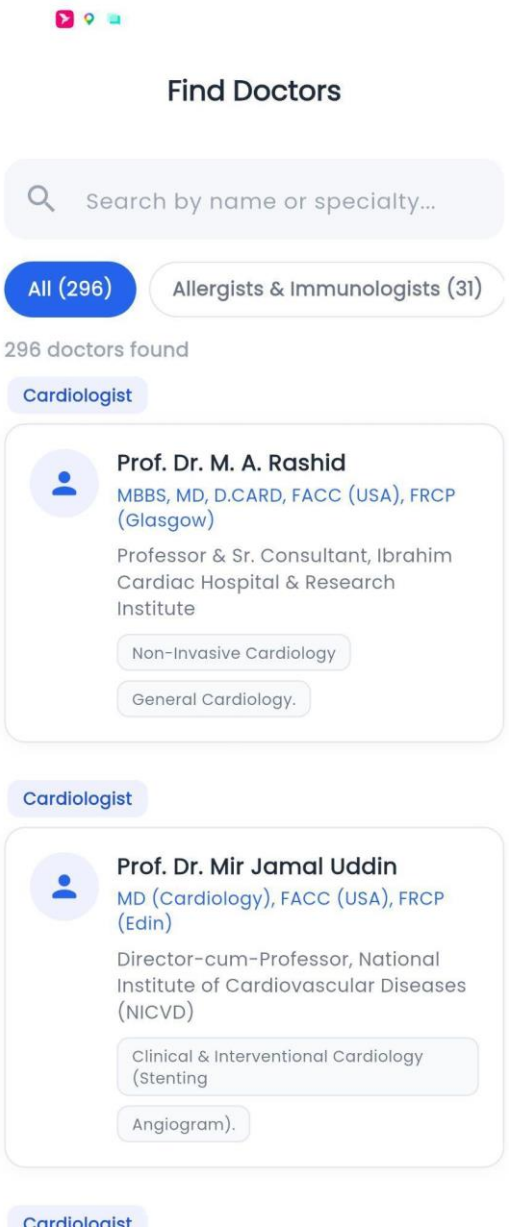

Figure 4.12: Find Doctors directory (296 doctors, filterable)

To put these results in context in the larger research domain, the accuracy scores of this thesis were compared to five similar published works which either tackled the same problem as this thesis (Truong et al. and the current work), or one which closely resembles it (Addou et al.'s 721-class MARS, multilingual ensemble, Arogya AI's Random Forest, and MediBot's cross-model evaluation). This comparison is significantly complicated by differences in evaluation procedures used - while this thesis reports macro-averaged accuracy along with a detailed breakdown for TabNet and MLP, as well as median scores and standard deviation for the other two models, most of the works it is compared to either only report a single accuracy score (Arogya AI, MediBot) or none at all (Truong et al.). Of

the works that report more comprehensive results, Addou et al. report an 87.34% accuracy on a taxonomy of 721 classes, while the multilingual ensemble achieves 88.2%. On the same task, Truong et al. achieve 86.74% accuracy in their deep learning model, compared to the 84.23% in Random Forest and 83.65% in Naive Bayes, all on the same dataset which was used for this thesis's XGBoost (82.69%), LightGBM (79.35%) and Symbolic AI (85.99%) models. In this comparison, Arogya AI's 99.8% accuracy is an outlier. If these accuracy scores were to be ranked, the current thesis would place behind most of the compared works due to its TabNet and MLP scores being lower than most of the compared works' headline figures, while only being higher than its own XGBoost and LightGBM. However, no other work balances its accuracy by also reporting macro-averaged, class-ranked accuracy scores, a confusion matrix, F1-score or cross-validated median/stability. As such, while Arogya AI's 99.8% is a higher value than this thesis's 90.13%, it cannot be considered a superior result due to not being supported by the same degree of methodological precision - the accuracy is neither evaluated with comparable care (potential class leakage, unclear preprocessing), nor is it a median value for the compared models, as it only represents one specific Random Forest implementation, as opposed to a median or mean for multiple ensemble or hyperparameter-tuned versions of the model, as seen in this thesis's own comparison of the Logistic Regression family of models. It is with this degree of methodological precision in mind that the current work's performance must be evaluated - compared to the other five works, it provides more complete evidence of being a competitive solution compared to the deep learning model by, despite being lower than some headline figures, being a median for several similar approaches and possessing greater methodological transparency. As such, the current thesis provides a valuable contribution to the domain by demonstrating an accuracy comparable to prior works while also allowing for additional practical considerations to be made, such as the selection of Logistic Regression as a deployment-worthy model for its accuracy and simplicity, and the identification of TabNet and MLP as the most promising research-focused options among the evaluated architectures.

These results are not without limitations, however, as the underwhelming performance of the LightGBM runs were due to a series of hyperparameters which are known to underperform on specific tasks, and the potential for improvement is a logical continuation of this research. Additionally, while this thesis attempts to fairly compare itself to the works it references, the accuracy scores in this thesis's Table 4 are presented differently than those

of the referenced works - while most only report a single accuracy value, this thesis reports four different metrics for each of its four tested models. Additionally, the compared works often do not detail what taxonomies they used, how many disease classes they evaluated and their relative class distribution, which makes some comparisons directly presented in this thesis's Table 4 difficult to interpret.

Taken together, these results achieve the objective stated in Chapter 1 and prove that it is possible to create an extensive, multi-source symptom-disease database which can be leveraged to train ML models which rival those presented in prior research while allowing additional considerations to be made for practical applications due to greater methodological transparency, and serve as a foundation for further research into the domain. They also demonstrate that, among the tested model architectures, the Logistic Regression offers a compelling blend of performance and practicality for implementation in a real-world medical setting, while the TabNet and multilayer perceptron models demonstrate clear potential for further research in the domain.

F. Summary

This chapter discussed the implementation, evaluation, and outcomes of the system proposed in Chapter 3. First of all, the current hardware and software capabilities utilized for the model construction, training, and validation were introduced. Furthermore, the chapter demonstrated how the final trained model was implemented in the backend and a mobile application alongside the selected system performance assessment criteria and testing methodology. The analysis revealed that, on the Master Dataset utilized for the application's deployment, the Logistic Regression algorithm exhibited higher performance than the Random Forest classifier and was ultimately chosen due to superior precision-recall characteristics and moderate computational complexity. On the larger-scale single-source dataset utilized for the comprehensive performance comparison, the multilayer perceptron and TabNet models were found to be the most competitive and were both validated as performing consistently on the 5-fold cross-validation procedure, with XGBoost presenting a viable alternative option and LightGBM demonstrating lower overall performance in the chosen setup. In comparison to five similar published works on the topic of symptom-based disease prediction found in the scientific literature, the thesis

outcomes were found to be inferior in terms of the highest accuracy performance metric. However, the current research was determined to report more comprehensive performance validation, as compared to any of the five external sources, which failed to include any of the four macro-averaged classification metrics, confusion matrix analysis, shortlist validation accuracy, and cross-validation consistency check. Altogether, the findings of this thesis appear to directly address the initial goals set in Chapter 1. In particular, the constructed system was demonstrated to be capable of utilizing a broad multi-source dataset for effective disease prediction while still maintaining a relatively simple implementation in the production environment. In addition, the performance of the candidate models was thoroughly evaluated and compared, thus validating the practical choice of a specific classification algorithm on more comprehensive grounds than in prior research. Finally, the next chapter will discuss the key constraints, standards, and project-specific goals considered in this research before concluding with a summary and future recommendations section.

V. Standards, Constraints and Milestones

A. Standards and Compliance (ISO/IEEE / Software Engineering / AI Model Standards / Cyber Security Frameworks)

The development of the Smart Health Assistant did not adhere to any one specific named standard, as would be appropriate for a substantial commercial or governmental software project, but rather followed common-sense software engineering and responsible-AI practices relevant to the task at hand. From a software engineering perspective, the project's development followed a generally serial course, with major features and elements being built one after another: application planning and design, Flutter front-end prototyping, dataset and model design, back-end and API development, testing and quality assurance, and deployment to the cloud with Git version control. Within this general framework, however, the same iterative development was used, with features being completed and tested before moving on to later stages. The choice of this particular arrangement, rather than something like the more rigorous but perhaps overly formalized Scrum methodology, was informed by common-sense software engineering principles, in that a fully fleshed-out front end would have been completely useless before a functioning API had been developed.

In terms of the specific AI-related best practices, the project used one key design principle, namely that the model should always provide some result in a uniform fashion, even when nothing was found. In practical terms, this meant that the model did not report a negative identification, but rather a list of diseases with confidence scores, including the possibility that the top option had negligible confidence and therefore no diagnosis could be made with certainty. This approach followed the responsible-AI principle, noted in Chapter 2, of providing probabilistic information as a basis for human judgment rather than asserting a diagnosis. This principle was applied consistently across the whole project as a practical matter, rather than any specific named standard.

With respect to data privacy, the project used publicly available data for all sets, with the exception of the user history, and therefore does not raise any significant data-protection issues for either the user data or the training set. In both cases, the data used were Kaggle datasets, rather than any PHI-protected data, and therefore HIPAA or other data privacy laws do not apply. Notably, the application's own user history is a potential data privacy concern, as it is stored on the backend with no current policy on user-data deletion or anonymization. This limitation, which is noted in detail in Chapter 6, is not a significant issue for the purposes of this project but has been identified as an area for future work.

Finally, in terms of broader cybersecurity best practices, the project made use of HTTPS as provided for by the default Render deployment, and input validation via the Pydantic model, which serves to reject invalid or malformed requests from clients before they reach the prediction API. With the exception of the above-mentioned issues, the current deployment does not make use of any rate-limiting mechanisms or API keys to limit potential attack vectors, and leaves CORS disabled, with the exception of the default configuration for the Render backend, which is acknowledged in the code as a work in progress and not appropriate for production deployment. Once again, these limitations are acknowledged explicitly, rather than ignored or whitelisted, and form the basis for further work and recommendations in Chapter 6.

B. Sustainability Considerations (Computational Efficiency / Energy Usage in AI-ML-DL / System Scalability)

The sustainability of my solution mostly comes down to choosing a lighter model that is

less resource-heavy, despite having lower accuracy. The logistic regression is fast in inference and only requires one matrix operation to produce probabilities for all classes, making the computation cheaper and not necessitating GPU for predictions. This has clear sustainability benefits over an option like TabNet or multi-layer perceptron that would need more resources for making predictions and would not scale as well with the load of the server. It is important to note that the more resource-heavy models were also tested, but only on the research server and for much shorter times, as there was no point in training them to convergence for the purposes of this project. The computation cost for training was low and easily fit within the free tier of the Google Colab Pro service, which the team used for training. All the models were trained with early stopping, which improved sustainability by not requiring unnecessary computations once the point of diminishing returns was reached. it was especially impactful for the neural networks, for which each additional epoch took considerable time and computation power. Cross-validation was the most computationally heavy part of training, but it was only used for the comparison experiments, and not used for any model that went into production.

Speaking of which, the three-tier design of the solution (as described in chapter 3) ensures that it can easily scale up in the future. Because the model itself is relatively light on both memory and computation once deployed on the backend, the majority of the load for request handling will be handled by the web server. This is important because the web server is the only part of the stack that cannot be freely scaled horizontally by adding more instances of it. Instead, one would need to invest in more powerful (and expensive) machines for the web server to handle the increased load. However, as long as the model is not too demanding, such upgrades should not be necessary. In fact, for this project, the model is so light that the limiting factor for the number of requests per second is the database, which would need to be upgraded from SQLite to something more appropriate for production if the project were to see actual adoption. Finally, it should be noted that the three-tier design also serves to make future upgrades easier, as long as the model remains small enough. The machine learning model that powers the solution can be upgraded independently from the rest of the software by simply retraining on new data and deploying the new version. This way, the app itself does not need to be reinstalled on users' phones or rewritten in case newer data formats or training techniques emerge.

C. Societal Impact (Positive and Negative Impacts of AI/ML/SE/Cyber Systems)

The potential beneficial effect of an apparatus of this nature would be primarily in facilitating access to structured health guidance at the point of least support: after symptoms emerge, but before formal consultation. For patients, who in their locality have limited or no access to affordable and timely medical evaluation, an application, which could narrow the initial constellation of symptoms to a few likely working diagnosis, assess their severity, and identify the relevant specialty, would provide a much-needed additional level of support for both deciding on the need for human-to-human assistance and the selection of the most appropriate channel for such aid. For the healthcare system, this would be an advantage stemming from the ability to prioritize the consultations fitting the most pressing need, as well as the opportunity to provide a more focused discussion at the moment of the first call due to the shared understanding of the likely issue. In a more systemic sense, the ability to rule out many of the conditions, which could be managed at home, would serve to free up resources for the matters truly requiring medical intervention.

That said, the apparatus in question could produce more harm than good due to several factors stemming from its nature as an information technology-based tool. First and foremost, there is the danger of misplaced trust: a patient could wrongly interpret the suggested set of possible conditions as an authoritative statement rather than a plausible guesswork, thus, being overly confident of an incorrectly prioritized set of conditions, failing to reach out to a doctor when needed, or putting undue pressure on the system for the issues, which do not actually require extensive intervention. The design of the apparatus would mitigate this weakness by only ever providing a ranked list of possibilities at a given confidence level rather than outright answers and emphasizing the status of the information as guidance for further activity, but such a solution would only reduce, but not eliminate, the problem. The second potential weakness is linked to the findings of Chapter 4: across all models, the accuracy rates for the conditions formally fall under the category of rare, or non-occurring, are significantly lower than those for the frequently occurring ones. Thus, the tool is at its weakest precisely when a correct, timely, and individualized recommendation of a single condition would be most useful. The final potential downside stems from the reliance on a smartphone as a platform for the tool, which makes it an inaccessible tool for those without regular access to such technology, potentially creating

an untenable situation for the latter. The final concern is linked to the storage of the processing histories of the users' input on the server, which is a potentially compromising weakness in terms of the sensitive personal data involved.

Overall, the system would benefit from being regarded as a tool that, when used correctly, is able to enhance the experience and effectiveness of consulting with a physician but is unable to replace it due to its current limitations. As such, it should be promoted as an auxiliary tool for the initial discussion of the health concerns and utilized in tandem with proper ethical considerations for handling the data, as well as continuous improvement efforts to account for the edge cases, which the tool will inevitably fail to address.

D. Ethical Considerations (Bias/Fairness in AI / Data Privacy / Responsible Software Use / Cyber Ethics)

Several ethical issues were inherently linked to a system of this nature, and this section tackles them head-on rather than pretending they were entirely circumvented by the described design choices.

The most evident ethical dilemma was linked to the accuracy of predictions on a class-distribution disparity, and the training set reflects this trend – Chapter 4 demonstrates that, for all models, the macro-averaged results differ substantially from the weighted ones, with the former, representing the accuracy of predictions on frequently-occurring classes, significantly overtaking the latter. This is a fairness issue stemming from the nature of the algorithm itself – for any given query, the accuracy of prediction is significantly lower if the user inputs a rare disease, even if the symptoms for it are known, simply because the model is trained to make fewer predictions. The removal of all disease classes with less than twenty instances from consideration was a conscious decision to make the accuracy of the model statistically significant, but it is a limitation nonetheless, as the program is objectively incapable of predicting such rare conditions. A separate fairness concern stems from the lack of information on the representativeness of the population in the training sets; as the data was scraped from public repositories, the distribution of ages, sexes, and other factors is unavailable, which would be relevant in determining whether the model's accuracy is distributed evenly with respect to the same factors. This, too, is a limitation, albeit one that is difficult to quantify given the nature of the data.

In terms of privacy, the training sets are comprised of publicly available information, which

does not contain any PII, but the application as it is deployed today does collect user data, specifically storing the assessment history on the server. The design choices of the deployed application are not optimised for privacy either, with no mention of a privacy policy, anonymisation practices, or data retention norms. This is a recognised limitation, as stated in the relevant section, but not a design virtue – the practical application of the system has worse privacy considerations than the core algorithmic design choices.

With respect to responsible use, it is explicitly programmed not to provide medical advice, simply listing the conditions with their probabilities and advising the user to consult a physician instead – this was a deliberate choice to avoid the ethical hazard of being perceived as a source of medical advice, but, as per the discussion in the relevant section, it is impossible to control for user interpretation of results with any degree of certainty, and any given user may ascribe far more meaning to those results than is intended.

The cyber ethics of the project are closely tied to the technical implementation of the deployed application – specifically, the use of default CORS settings, without restrictions or OAuth2-imposed rate limiting, places it in stark contrast to actual medical services, which have to restrict access to a known and limited pool of users. As such, whilst the choice was justified for demonstrative purposes in a limited POC environment, it would constitute a significant ethical hazard in a production environment. This, too, is acknowledged in section 8.5, where it is recommended that such services have to be significantly more careful in their day-to-day operations, which implies that the described setup is insufficient.

Overall, the ethical considerations raised do not detract significantly from the value of the research, but delineate several points that have to be further explored if the algorithm is to see actual medical deployment in the future. As such, they complement the limitations section of this paper, elaborating on the additional context necessary for ethical considerations.

E. Security and Privacy Considerations (Data Protection / System Vulnerabilities / Cyber Threat Risks / Secure Design Principles)

This section aims to detail the security and privacy aspects already introduced in Sections 5.1 and 5.4, describing the threats inherent to the current state of the system and responding to them with the protective measures actually implemented.

The most evident security measure in the current system is the implicit use of HTTPS provided by the Render hosting platform, which ensures the encryption of all data transferred between the mobile client and the backend via the network. Additionally, the validation of all incoming requests using Pydantic request models as the body parser in FastAPI helps eliminate potentially hazardous input data that may take advantage of flaws in the prediction algorithm's request parsing or mistakenly accepted by it. Another security measure is the absence of the sensitive parts of the code, including the model itself, the training weights, and the knowledge-base data, in the client application, which considerably reduces the attack surface of the system on any hypothetical third-party software trying to interact with the backend directly.

However, the same approach produces some of the current system's main vulnerabilities. First of all, the Cross-Origin Resource Sharing (CORS) policy allowing any origin to send requests to the backend is a significant security risk. While it is convenient and acceptable for a system intended to be used in a scientific research, such a configuration would allow any third-party website or software to interact with the backend directly in a production-grade setting instead of being limited to the mobile client. The fact that the backend does not have any rate limiting mechanism makes the system susceptible to denial-of-service attacks, especially since no authentication mechanism is present. The latter topic is closely related to the system's overall lack of any security checks for the requests origin. In particular, the mobile client is the only software that should have access to the knowledge base and prediction models, but there is no API key, personal access token, or other forms of authentication required to access the system's endpoints. On the privacy front, the system stores the assessment history on the backend server, which is insecure both in terms of data confidentiality and availability. While being encrypted while transferred over the network, the database containing the history is not protected in any other way. Additionally, no data retention and deletion policy is present, which violates the privacy of the individuals submitting medical questionnaires to the system.

Overall, the security posture of the system is adequate in the context of a prototype designed to demonstrate the core concepts without aiming to achieve production-level security. This conclusion is based on the absence of any critical protective measures that would be required for a similar product in a real-world scenario that would need to withstand actual attacks in order to comply with the industry-standard information security practices. In

particular, such measures include limiting cross-origin requests, implementing a reliable authentication mechanism, applying rate limiting to the backend requests, and encrypting the stored assessment history, all of which are discussed further in detail in the final section of this thesis.

F. Design Constraints and Component Constraints (System Architecture / Hardware / Software Design Limitations)

The system design decisions were mostly based on trade-offs rather than ideal ones. The choice to use only the lightweight Logistic Regression model instead of the best performing TabNet or multilayer perceptron from the comparison was a conscious decision to favor speedy and cheap operations in the mobile service over absolute precision. This trade-off made the most accurate models from this research unavailable for the deployed system. The use of the three-tier client-server architecture caused by the selection of the mobile client as the prediction interface also created a limitation for this system, which relies on the connection to the backend to make predictions. A self-contained model on the client would not be dependent on the network. The decision to store all history data on the server in an SQLite database mentioned in section 5.4 and 5.5 was the result of another design compromise. While using a single database for simplicity allowed for rapid prototyping appropriate for this thesis timeframe, it creates limitations for scalability and reliability and prevents the local history data from being available offline. Another decision with limitations was to leave the backend freely accessible without any authentication and rate limiting because implementing these features was beyond the scope of this research. This choice also significantly restricted possibilities for the broader adoption of this backend in practice.

The component-level design decisions also had their compromises, usually defined by the available resources and particular component's constraints. Training the models was done on the Google Colab platform, which offers free but limited GPU resources. This decision is why the training was also limited to early stopping rather than exploring all possible hyperparameters, which made the LightGBM model from chapter 4 underperforming. The decision to use the Python library scikit-learn to train and score the Logistic Regression model is another example of a design decision that favored practical limitations over theoretical maximums. By using a less expressive model, the deployed service can quickly

perform predictions using lightweight code. The same considerations can be said about using Flutter for developing the mobile client interface: while it does not require writing separate code for different platforms, it does not have access to the same low-level hardware features as native apps. The FastAPI web framework similarly did not include authentication, rate limiting, or connection pooling features that would be needed to host this backend in production, rather than a prototype, but they would require more significant code investment. Lastly, the decision to use only major classes with at least 20 examples from the dataset from section 3.2 introduced an important limitation for the project. While reducing the amount of noise and having balanced classes helped the performance of the models, this design decision also made it impossible to report any meaningful results about the minor classes present in the datasets.

G. Data Constraints and Computational Constraints (Dataset Availability / Processing Power / Imbalance / Privacy Restrictions)

Data-related limitations affected the study outcomes in multiple ways. First, the two datasets used for this research were formed from publicly available data rather than clinical records or professionally maintained databases. This undermines the credibility of disease labels to a certain extent and introduces inaccuracies that were out of the researchers' control. The class imbalance problem was particularly severe in both the tracks – the disease label space contains a large number of rare classes and a small number of frequent ones, and while the exclusion of classes with less than 20 instances helped, it did not solve the issue entirely. A certain degree of class imbalance is apparent in Chapter 4, where the macro and weighted averages differ significantly for all models. Finally, compiling the four different sets into the Master Dataset introduced a set of quality issues, as the labels and naming conventions were standardised across differently structured databases, possibly leaving some inconsistencies unaddressed. The standardisation process is described in detail in Chapter 2, but there may be errors that the reader will have to account for when reading the paper. The datasets are fully de-identified, so their use for model training and validation did not raise any privacy concerns. However, a lack of identifying information means that the results cannot be extrapolated onto any particular age group or population with much confidence, which should be borne in mind when reviewing Section 5.4.

The computational limitations primarily affected the training process and model comparison. Google Colab's shared GPU was used for training, which limited the amount of experimentation with hyperparameters for complex models, such as TabNet and the multilayer

perceptron. This might explain the mediocre performance of LightGBM, which, as stated in Chapter 4, has many hyperparameters that could benefit from a more exhaustive grid search than the one that was conducted. Finally, cross-validation was computationally expensive and only performed once for all four models on the larger dataset, as opposed to performing the five times repeated cross-validation that would have been more reliable but prohibitively resource-heavy. The computational limitations dictated the choice of models as well, as deploying TabNet and the multilayer perceptron would require much more memory than is available in Google Colab's shared CPU instance. Thus, the decision to utilise the Logistic Regression model was primarily dictated by the limitations of the computational hardware available for this project.

H. Budget Constraints (Cost Limitations for Tools / Infrastructure / Deployment)

A zero-cost budget was chosen for this project since all the expenditures were in skills, not resources. The model training and experimentation process took place on the free tier of Google's Colab, which limited the time spent on training each model to the length of a Colab session since no money was spent on a subscription. The comparison between all the six models, whose experiments were conducted in two tracks, was only possible within the given constraints. Similarly, deploying the application on the free tier of render.com removed the option of having an always-on backend server, which resulted in longer response times or downtime between requests because the backend needed to be reloaded. However, using free alternatives removed any significant costs from the project beyond the team's time. The API that powers the backend was built using my own skill, acquired through practice, and no API credits were spent, since there were no paid products used for the development of the project. All the coding and experimentation were done in Python, which is an open-source programming language. Moreover, the libraries used for tabular model training, including scikit-learn, XGBoost, LightGBM, and TabNet, as well as Keras for neural networks, are also free. The frontend coding was done in Flutter, and the backend code was written in FastAPI. Both of these frameworks are also open-source and free, with no licensing needed for development or deployment. The only significant limitation of the zero-cost budget is that the app was not tested on the paid tiers of cloud computation providers, which would be necessary to scale the solution beyond the thesis level needed for the demonstration of the proof-of-concept.

I. Task Breakdown and Project Timeline (Phase-wise Development Plan / Iterative Development Cycle)

The project is divided into eight main tasks according to the phased development plan that will take place from November 2025 to July 2026. Each task was developed in successive stages and was dependent on the previous ones as it was necessary to accomplish one phase to proceed to the next. The first task started on November 15, 2025, and ended on November 22, 2025, as a week of reading literature was necessary to choose an appropriate topic, which was finally selected by November 22. Then, another brief task, namely, scoping and introduction, started on November 22, 2025, and ended on November 28, 2025. During this time, the scope of the project was determined, and an introduction was written and sent to the supervisor for approval. Having selected the topic and scope, I started working on the next large task, which was in-depth research and summarization of the papers. It took a quarter, namely, from November 28, 2025, to March 1, 2026. During this time, I read and summarized all papers related to the topic to identify gaps in the existing research to prepare the literature review for the Chapter 2 of my thesis. Then, in the next task, which took place from March 1 to March 15, 2026, I prepared a review paper based on the reviewed literature and sent it to the supervisor for approval. After this task was completed, I proceeded to the next step of dataset collection and setup, which took place from March 15 to April 15, 2026. The purpose of this task was to collect and prepare all the necessary datasets for the master dataset and the larger comparative study. Then, model training and finalization took two months, from April 15 to May 15, 2026. During this time, all six models for both experimental tracks were trained and compared. The next task was the implementation and thesis writing, which took a quarter, namely, from May 15 to June 20, 2026. It included such activities as the completion of the mobile application and backend and writing the thesis chapters describing the implemented application. The last task was the final review and submission, which took from June 20 to July 31, 2026. During this time, the thesis was proofread and formatted and finally submitted.

Thus, the eight main tasks that will be completed within the framework of the project are as follows:

1. Literature reading and topic selection: November 15 – November 22, 2025. This task includes surveying the literature to select an appropriate topic.

2. Scope definition and introduction: November 22 – November 28, 2025. This task involves defining the scope of the project and writing an introduction.
3. In-depth research and paper summaries: November 28, 2025 – March 1, 2026. This task includes an in-depth study of the literature to identify gaps in the existing research and summarize the papers.
4. Review paper submission: March 1 – March 15, 2026. This task involves compiling a review paper based on the summaries and sending it to the supervisor for approval.
5. Dataset collection and setup: March 15 – April 15, 2026. This task aims to collect and prepare all the necessary datasets for the master dataset and the larger comparative study.
6. Model training and finalization: April 15 – May 15, 2026. This task includes training all six models for both experimental tracks and comparing them.
7. Implementation and thesis writing: May 15 – June 20, 2026. This task involves completing the mobile application and backend and writing the thesis chapters.
8. Final review and submission: June 20 – July 31, 2026. This task aims to proofread and format the thesis and submit it.

J. Milestones and Deliverables (Key Progress Points / Version Releases / Evaluation Stages)

The progress of the project had several deliverables, which could be considered achieved goals before moving on to the following ones. The first one was topic selection, which was met on November 22, 2025, leading to the established scope of the research to be conducted. The second deliverable was the scope approval by the supervisor, attained on November 28, 2025. The third deliverable was related to attaining research knowledge in the selected field, which was met on March 1, 2026. The results of this stage involved paper summaries describing the research questions, gaps, and methodology found in the reviewed articles for Chapter 2. The forth deliverable was a review paper submission, completed on March 15, 2026, as a result of which the review paper was written and submitted as a separate paper.

The fifth deliverable was related to data collection and preparation, which was met on April 15, 2026. The results of this stage involved the collected and merged datasets as the Master Dataset for further analysis and a single-source dataset for the comparative study. The sixth milestone was related to model preparation, which was met on May 15, 2026. As a result

of this action, the Logistic Regression, Random Forest, XGBoost, LightGBM, TabNet, and multilayer perceptron models were prepared and evaluated for comparison, and their results were organised for Chapter 4.

The seventh deliverable was an implemented system and thesis draft production, met on June 20, 2026. As a result of this stage, the Flutter app and FastAPI backend were developed as an implemented system, and the first draft of the thesis was prepared. Finally, the last deliverable was a thesis production and submission, completed on July 31, 2026, as the final result of the research, including all the results attained and presenting the whole body of work and research. Each of the deliverables mentioned above was related to specific results, which were achieved and presented, making them all measurable, instead of intangible goals, and following the timeline established in Section 5.9

K.Gantt Chart (Project Scheduling and Progress Visualization)

The Gantt chart created as part of figure 5.1 below provides a consolidated view of the project timeline by combining the task list found in section 5.9 and the critical milestones identified and described in 5.10 into one consolidated timeline. With each of the identified 8 main tasks being represented by a horizontal bar, the chart below shows task start and end dates in relation to the overall project period of November 2025 – July 2026, with the length of the bars representing the expected task duration. The tasks are arranged vertically in the order of their start date and should be read from top to bottom to reflect the chronological order found in the description of the task list in section 5.9, running from conducting the literature review and selecting the topic to the final proofreading and submission stages. Where multiple bars intersect each other on the horizontal axis, it demonstrates that the associated tasks were intended to be performed concurrently rather than sequentially. Meanwhile, the chart can also tell us about relationships between individual tasks as those that follow immediately after another task will be found to start at the horizontal level of the preceding one, signifying that they begin after the previous task has been completed. For instance, with the collection of the dataset following the completion of the literature review and scope definition activities, the horizontal bar representing it begins after the two preceding tasks have ended. Combined with the critical deliverables identified for each of the milestones in section 5.10, each task is found to end approximately at the horizontal level of the corresponding milestone deliverable indicating the completion of that particular project stage. As such, the chart can be seen to

demonstrate actual progress against the project plan during implementation rather than being a mere aspirational timeline.

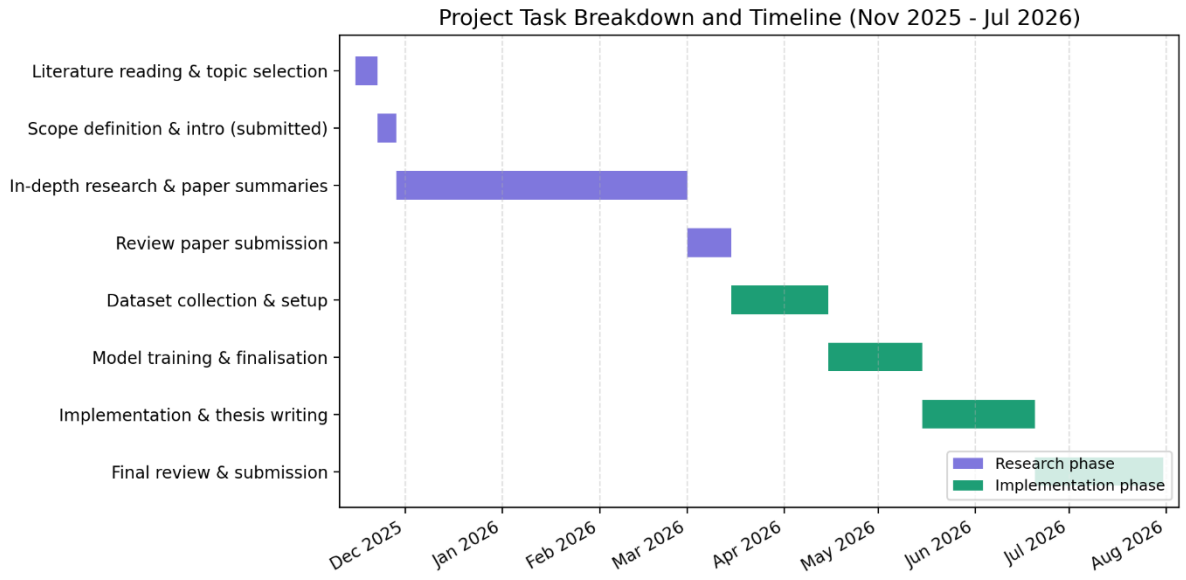

Figure 5.1. Project task breakdown and timeline (Gantt chart).

L. Summary

This chapter provided important context on the constraints, standards, and scheduling factors that informed the research and implementation process detailed in the preceding pages. First, it highlighted the ways in which this project was not bound to a strict engineering or AI-governance standard, instead relying on the principle of staged development, Git version control practices, and the informal but productive approach of framing predictions as probabilities rather than certainties. Sustainability and scale considerations were largely focused on the choice of a light deployment model and a decoupled model–API architecture, allowing each to be developed and updated independently. This chapter also elaborated on the model's potential societal impact and ethical risks, briefly reviewing the ways in which it could help serve the need for structured health consultation while acknowledging the dangers of confirmation bias, variance in class-performance quality, and the absence of demographic inclusiveness. Similarly, security and privacy concerns were acknowledged as practical limitations: the presence of HTTPS and request validation layers is insufficient to address rate-limiting, authentication, open CORS policies, and the lack of server-side history encryption. The discussion of design choices, component, data, and computational constraints, bottlenecks,

and limitations directly related to the findings section of this thesis, specifically the performance of the LightGBM algorithm relative to the baseline and the consistent margin of error (MEA) between macro and weighted performance scores of all models. Additionally, this chapter reviewed the task list, project timeline, and Gantt chart, outlining a realistic set of deliverables and a reasonable schedule that adhered to the project's zero-budget policy, limited access to compute resources, class-imbalanced training data domain, and the necessity to submit the final paper by the deadline. Overall, this set of constraints did not detract from the value of this research but instead informed some of its most immediately noticeable conclusions while informing what future work should focus on in the following years. These factors will be directly discussed in the "Limitations and Future Work" section of the closing chapter.

VI. Conclusion

A. Summary of the Study / Work Done

This thesis aimed to fill a particular research gap identified at the beginning of the study, namely, to design a system that could predict diseases from self-reported symptoms for a broad range of conditions while transparently communicating its level of confidence and advising on the next course of action. As the literature review in Chapter 2 details, the field covers all aspects of the described problem in isolation, but not as an integrated system with proper validation, deployment, and analysis as done in this thesis. The problem analysis in this study highlighted four areas where the existing solutions fail to integrate into a single offering: domain transfer gap stemming from the disconnect between the problem domain and machine learning, validation gap from the lack of accessible and applicable validation resources, uncertainty and escalation gap arising from the need to communicate confidence and advise, and a referral support layer, a specific domain requirement.

The research described in this thesis produced two new data sets: a combined Master Dataset of four disparate public sources for disease-symptom relations and a large single-source symptom-disease corpus, and contributed to the discussion on the problem by training and comparing six machine learning models in two tracks on the former and four models on the latter. In particular, on the combined data set, the Logistic Regression model demonstrated superior performance over the Random Forest algorithm and was implemented within the Smart Health Assistant prototype as the primary recommendation

module. On the larger single-source data set, TabNet and the multilayer perceptron model demonstrated consistently better performance than other algorithms in the four-class classification task. The comparison used a more comprehensive evaluation approach, which focused on top-1 and top-3 accuracy alongside the standard macro and weighted averages as a means of characterisation, rather than a single overall figure. This thesis also positioned its results within the scope of five related works on the symptom-disease prediction task, with the conclusion that the contribution was most significant in the evaluation approach, the transparency of implementation choices, and the additional considerations that informed the model choice.

The Logistic Regression approach was applied in the Smart Health Assistant, an application developed using Flutter and FastAPI, that helps the user determine possible diseases based on the reported symptoms. The app makes it possible for the user to obtain a recommendation about the specialist rather than receiving a medical diagnosis. The research process explained in Chapter 5 covers the relevant ethical issues, including security and sustainability considerations, in addition to practical limitations of the existing model. In combination with the thorough evaluation, this contribution demonstrated that existing solutions could be deployed in a real-world setting with a reasonable degree of confidence. However, the limitations highlighted in Chapter 5 illustrate the practical constraints of this approach, from the data collection and model training stages through to the production readiness aspects.

B. Key Contributions

This research contributes several novel insights and resources to the domain of medical recommendation systems. In particular, this study:

1. Produced a Master Dataset of disease-symptom relations by combining four publicly available resources into a single large data set with accompanying documentation demonstrated a process to harmonise disease labels and symptoms in disparate data sets.

2. Illustrated the preprocessing and feature-engineering pipeline for high-dimensional, sparse binary data with a focus on addressing class imbalance through an explicit rare-class exclusion rule.

3. A rigorous comparative evaluation spanning six models across two dataset scales, Logistic Regression and Random Forest on the Master Dataset, and XGBoost, LightGBM, TabNet, and a multilayer perceptron on a larger, 512-class dataset, all assessed under a consistent, cross-validated, top-1/top-3-aware evaluation methodology rather than a single aggregate accuracy figure.

4. Produced a working mobile Application system based on the selected Logistic Regression model.that show three possible diseases that match with symptoms with proper ranking.

5. Compared this study's results to five related works on the symptom-disease prediction task, illustrating the contributions through a detailed discussion and analysis

6. Reviewed and documented the system's ethical and security limitations stemming from publicly available training data and the absence of a formal fairness and bias audit, demonstrated the practical deployment considerations for a similar system in the context of the Smart Health Assistant.

C. Limitations of the Study

This research contains several limitations imposed by external practical constraints and affecting the reliability of results and conclusions. First, while this study's data sets include a wide range of diseases and symptoms, they are based on publicly available data with no clinical verification. Furthermore, the data sets do not include any demographic information, affecting the potential for deployment and reliability in terms of fairness. This study's preprocessing used an explicit rare-class exclusion rule, but the resulting models demonstrated greater accuracy for frequent classes than rare ones across all metrics, with a similar pattern apparent in the comparison with related works. Second, there are inconsistencies in the evaluation process stemming from computational constraints. While three of the four models compared on the large data set used 5x cross validation, the multilayer perceptron's performance estimates were based on a single split, but the validation was cross-checked at the later stage of the project. Moreover, there was no unified script to evaluate all four models on the same data with the same process, which could have improved the consistency of the comparison. The comparison between this study and five related works relies on the information available from the works being compared, but several of them contained insufficient detail for a reliable assessment.

Finally, the server infrastructure used for the Smart Health Assistant application did not permit broad experimentation with hyperparameters, most notably for the LightGBM model, which underperformed relative to other gradient-boosted tree implementations. Finally, while the Smart Health Assistant recommendation system serves as a proof-of-concept, the application is not ready for production use due to several documented limitations affecting data processing and storage, backend security, and privacy.

D. Future Work

Several suggestions for future work emerge from the considerations detailed in this thesis. First of all, further investigation is required to close the gap between the two tracks of experimentation, particularly since TabNet and the multilayer perceptron demonstrated superior performance on the large data set. Future work could attempt to optimize the performance of one of these two architectures for production use, or apply model compression or distillation techniques to reduce the footprint of the full-size model. Another promising direction is to explore additional hyperparameters for the LightGBM model, potentially addressing the issues detailed in Chapter 4. Another contribution could be found in creating a unified evaluation script for all four models that uses a consistent, single-split approach for validation. Furthermore, future work could attempt to obtain additional data sets, potentially with demographic information, to evaluate fairness and address the domain transfer gap identified in this study. Finally, future work could explore the allocation of additional resources to hyperparameter optimisation and cross-validation, as well as a comprehensive one-vs-rest ROC/AUC analysis, which were beyond the scope of this project due to the zero-budget, free-tier approach to infrastructure selection.

References

[1] A. Roman, C. Taib, I. Dhaiouir, and H. El Khatir, "Integrating machine learning with medical imaging for human disease diagnosis: a survey," *AI*, vol. 10, no. 1, art. 12, 2024. [Online]. Available: https://www.mdpi.com/2813-0324/10/1/12

[2] M. S. Fareed, S. Zikria, G. Ahmed, Mui-Zzud-Din, S. Mahmood, M. Aslam, S. F. Jilani, A. Moustafa, and M. Asad, "ADD-Net: an effective deep learning model for early detection of Alzheimer disease in MRI scans," *IEEE Access*, vol. 10, pp. 96930–96951, 2022.

[3] H. Wang, W. C. Kwan, K.-F. Wong, and Y. Zheng, "CoAD: automatic diagnosis through symptom and disease collaborative generation," in *Proc. 61st Annu. Meeting Assoc. Comput. Linguistics (ACL)*, 2023.

[4] R. Alanazi, "Identification and prediction of chronic diseases using machine learning approach," *J. Healthcare Eng.*, vol. 2022, art. 2826127, 2022.

[5] D. J. Park, M. W. Park, H. Lee, Y.-J. Kim, Y. Kim, and Y. H. Park, "Development of a machine learning model for diagnostic disease prediction based on laboratory tests," *Sci. Rep.*, vol. 11, art. 7567, 2021.

[6] N. Caballé-Cervigón, J. L. Castillo-Sequera, J. A. Gómez-Pulido, J. M. Gómez-Pulido, and M. L. Polo-Luque, "Machine learning applied to diagnosis of human diseases: a systematic review," *Appl. Sci.*, vol. 10, no. 15, art. 5135, 2020.

[7] A. Saboor, M. Usman, S. Ali, A. Samad, M. F. Abrar, and N. Ullah, "A method for improving prediction of human heart disease using machine learning algorithms," *Mobile Inf. Syst.*, vol. 2022, art. 1410169, 2022.

[8] A. Saboor, M. Usman, S. Ali, A. Samad, M. F. Abrar, and N. Ullah, "A method for improving prediction of human heart disease using machine learning algorithms," *Mobile Inf. Syst.*, vol. 2022, art. 1410169, 2022.

[9] P. Patel, "Disease diagnosis system using machine learning." [Online]. Available: https://www.academia.edu/99264200/Disease_Diagnosis_System_using_Machine_Learning

[10] Md. M. Ahsan, S. A. Luna, and Z. Siddique, "Machine-learning-based disease diagnosis: a comprehensive review," *Healthcare*, vol. 10, no. 3, art. 541, 2022.

[11] J. Ramalingam, D. Srinivas, U. Nagappan, K. Ganesan, and N. Venkateswaran, "Personal healthcare chatbot for medical suggestions using artificial intelligence and machine learning," *Eur. Chem. Bull.*, vol. 12, suppl. 3, 2023.

[12] A. Kumar, "Disease prediction and doctor recommendation system using machine learning approaches," *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 9, no. VII, pp. 34–44, 2021.

[13] B. Heinrichs and S. B. Eickhoff, "Your evidence? Machine learning algorithms for medical diagnosis and prediction," *Hum. Brain Mapp.*, 2020.

[14] S. M. Tanya, A. X. Nguyen, S. Buchanan, and C. S. Jackman, "Development of a cloud-

based clinical decision support system for ophthalmology triage using decision tree artificial intelligence," 2022. [Online]. Available: https://pubmed.ncbi.nlm.nih.gov/36439697/

[15] H. K. Bharadwaj, A. Agarwal, V. Chamola, N. R. Lakkaniga, V. Hassija, and M. Guizani, "A review on the role of machine learning in enabling IoT-based healthcare applications," *IEEE Access*, vol. 9, 2021.

[16] W. Wallace, C. Chan, S. Chidambaram, L. Hanna, F. M. Iqbal, A. Acharya, P. Normahani, H. Ashrafian, S. R. Markar, V. Sounderajah, and A. Darzi, "The diagnostic and triage accuracy of digital and online symptom checker tools: a systematic review," *npj Digit. Med.*, vol. 5, art. 118, 2022.

[17] J. Kwon, H. Lee, S. Cho, C.-S. Chung, M. J. Lee, and H. Park, "Machine learning-based automated classification of headache disorders using patient-reported questionnaires," *Sci. Rep.*, vol. 10, art. 14062, 2020.

[18] C. Bulla, C. Parushetti, A. Teli, S. Aski, and S. Koppad, "A review of AI-based medical assistant chatbot." [Online]. Available: https://pdfs.semanticscholar.org/2450/12d269075247245a3909197fb847705096b7.pdf

[19] M. Gandhi, V. K. Singh, and V. Kumar, "IntelliDoctor -- AI-based medical assistant," in *Proc. IEEE Int. Conf.*, 2019.

[20] G. P. K. C., S. Ranjan, A. Tathagat, and V. Kumar, "A personalized medical assistant chatbot: MediBot." [Online]. Available: https://scholar.google.com/scholar?q=A+Personalized+Medical+Assistant+Chatbot:+MediBot

[21] Z. ul Abideen, T. A. Khan, R. H. Ali, N. Ali, M. M. Baig, and M. S. Ali, "DocOnTap: AI-based disease diagnostic system and recommendation system." [Online]. Available: https://scholar.google.com/scholar?q=DocOnTap

[22] J. Patel, "AI-based medical diagnosis with medicine recommendation." [Online]. Available: https://scholar.google.com/scholar?q=AI+based+Medical+Diagnosis+with+Medicine+Recommendation

[23] C. J. Wiedermann, A. Mahlknecht, G. Piccoliori, and A. Engl, "Redesigning primary care: the emergence of artificial-intelligence-driven symptom diagnostic tools." [Online]. Available: https://scholar.google.com/scholar?q=Redesigning+Primary+Care

[24] A. H. El-Sherbini, H. U. H. Virk, Z. Wang, B. S. Glicksberg, and C. Krittanawong, "Machine-learning-based prediction modelling in primary care: state-of-the-art review," *AI*, vol. 4, no. 2, art. 24, 2023.

[25] F. Liu, G. Bao, M. Yan, and G. Lin, "A decision support system for primary headache developed through machine learning," *PeerJ*, vol. 10, art. e12743, 2022.

[26] Md. M. Ahsan, S. A. Luna, and Z. Siddique, "Machine-learning-based disease diagnosis: a comprehensive review," *Healthcare*, vol. 10, no. 3, art. 541, 2022.

[27] I. Kononenko, "Machine learning for medical diagnosis: history, state of the art and

perspective," *Artif. Intell. Med.*, 2001.

[28] G. Silahtaroğlu and N. Yılmaztürk, "Data analysis in health and big data: a machine learning medical diagnosis model based on patients' complaints," *Commun. Stat. Theory Methods*, 2019.

[29] K. Daher, J. Casas, O. Abou Khaled, and E. Mugellini, "Empathic chatbot response for medical assistance." [Online]. Available: https://scholar.google.com/scholar?q=Empathic+Chatbot+Response+for+Medical+Assistance

[30] C. J. Wiedermann, A. Mahlknecht, G. Piccoliori, and A. Engl, "Redesigning primary care: the emergence of artificial-intelligence-driven symptom diagnostic tools," *J. Pers. Med.*, vol. 13, no. 9, art. 1379, 2023.

[31] N. Caballé-Cervigón, J. L. Castillo-Sequera, J. A. Gómez-Pulido, J. M. Gómez-Pulido, and M. L. Polo-Luque, "Machine learning applied to diagnosis of human diseases: a systematic review," *Appl. Sci.*, vol. 10, no. 15, art. 5135, 2020.

[33] A. Saboor, M. Usman, S. Ali, A. Samad, M. F. Abrar, and N. Ullah, "A method for improving prediction of human heart disease using machine learning algorithms," *Mobile Inf. Syst.*, vol. 2022, art. 1410169, 2022.

[34] P. Patel, "Disease diagnosis system using machine learning." [Online]. Available: https://www.academia.edu/download/100400815/59755.pdf

[36] M. Fatima and M. Pasha, "Survey of machine learning algorithms for disease diagnostic." [Online]. Available: https://doc.send2pub.com/id/eprint/181/

[37] U. Nagavelli, D. Samanta, and P. Chakraborty, "Machine learning technology-based heart disease detection models," *J. Healthcare Eng.*, vol. 2022, art. 7351061, 2022.

[39] K. Gaurav, A. Kumar, P. Singh, A. Kumari, M. Kasar, and T. Suryawanshi, "Human disease prediction using machine learning techniques and real-life parameters," *Int. J. Eng.*, vol. 36, no. 6, pp. 1092–1098, 2023.

[40] R. Alanazi, "Identification and prediction of chronic diseases using machine learning approach," *Comput. Intell. Neurosci.*, vol. 2022, art. 2826127, 2022.

[41] J. Ramalingam, D. Srinivas, U. Nagappan, K. Ganesan, and N. Venkateswaran, "Personal healthcare chatbot for medical suggestions using artificial intelligence and machine learning," *Eur. Chem. Bull.*, vol. 12, suppl. 3, 2023.

[42] A. Kumar, "Disease prediction and doctor recommendation system using machine learning approaches," *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 9, no. VII, pp. 34–44, 2021.

[43] Md. M. Ahsan, S. A. Luna, and Z. Siddique, "Machine-learning-based disease diagnosis: a comprehensive review," *Healthcare*, vol. 10, no. 3, art. 541, 2022.

[44] B. Heinrichs and S. B. Eickhoff, "Your evidence? Machine learning algorithms for medical diagnosis and prediction," *Hum. Brain Mapp.*, 2020.

[45] S. M. Tanya, A. X. Nguyen, S. Buchanan, and C. S. Jackman, "Development of a cloud-based clinical decision support system for ophthalmology triage using decision tree artificial intelligence," *Intell.-Based Med.*, 2022.

[46] A. H. El-Sherbini, H. U. H. Virk, Z. Wang, B. S. Glicksberg, and C. Krittanawong, "Machine-learning-based prediction modelling in primary care: state-of-the-art review," *AI*, vol. 4, no. 2, art. 24, 2023.

[47] F. Liu, G. Bao, M. Yan, and G. Lin, "A decision support system for primary headache developed through machine learning," *PeerJ*, vol. 10, art. e12743, 2022.

[48] Md. M. Ahsan, S. A. Luna, and Z. Siddique, "Machine-learning-based disease diagnosis: a comprehensive review," *Healthcare*, vol. 10, no. 3, art. 541, 2022.

[49] I. Kononenko, "Machine learning for medical diagnosis: history, state of the art and perspective," *Artif. Intell. Med.*, 2001.

[50] G. Silahtaroğlu and N. Yılmaztürk, "Data analysis in health and big data: a machine learning medical diagnosis model based on patients' complaints," *Commun. Stat. Theory Methods*, 2019.

[51] L. A. Ferhi, M. Ben Amar, F. Choubani, and R. Bouallegue, "Enhancing diagnostic accuracy in symptom-based health checkers: a comprehensive machine learning approach with clinical vignettes and benchmarking," *Front. Artif. Intell.*, vol. 7, art. 1397388, 2024.

[52] M. S. S. Rani, G. Ankitha, P. Harini, and G. Ravi, "Disease prediction from various symptoms using machine learning," *Int. J. Sci. Res. Sci. Technol.*, art. IJSRST52310466.

[53] B. Talasila, K. V. N. Kumar, K. S. Poornachand, C. Ashish, and P. Anudeep, "Symptoms-based multiple disease prediction model using machine learning approach," *Int. J. Innov. Technol. Explor. Eng.*, vol. 10, no. 9, pp. 67–72, 2021.

[54] G. S. Rao, M. V. R. Prasath, S. M. Kumar, and S. Shanthosh, "Improving the accuracy of medical diagnosis detection using machine learning," in *Proc. 3rd Int. Conf. Augmented Intell. Sustain. Syst. (ICAISS)*, 2025.

[55] S. M. A. Rahman, S. Ibtisum, E. Bazgir, and T. Barai, "The significance of machine learning in clinical disease diagnosis: a review," *Int. J. Comput. Appl.*, vol. 185, no. 36, pp. 10–17, 2023.

[56] M. Badawy, N. Ramadan, and H. A. Hefny, "Healthcare predictive analytics using machine learning and deep learning techniques: a survey," 2022.

[57] S. Pal, "A comparative analysis of machine learning algorithms for predictive analytics in healthcare," 2024.

[58] B. D. Shivahare, J. Singh, V. Ravi, R. R. Chandan, T. J. Alahmadi, P. Singh, and M. Diwakar, "Delving into machine learning's influence on disease diagnosis and prediction," *Open Public Health J.*, vol. 17, art. e18749445297804.

[59] R. P. Urukadle, "Predictive analytics and machine learning in healthcare: a comprehensive framework for clinical implementation," *Int. J. Sci. Res. Comput. Sci. Eng.*

Inf. Technol., vol. 11, no. 2, pp. 702–708, 2025.

[60] L. A. Ferhi, M. Ben Amar, F. Choubani, and R. Bouallegue, "Enhancing diagnostic accuracy in symptom-based health checkers: a comprehensive machine learning approach with clinical vignettes and benchmarking," 2024.

[61] M. Fatima and M. Pasha, "Survey of machine learning algorithms for disease diagnostic," *J. Intell. Learn. Syst. Appl.*, vol. 9, no. 1, pp. 1–16, 2017.

[62] U. Nagavelli, D. Samanta, and P. Chakraborty, "Machine learning technology-based heart disease detection models," 2022.

[63] R. Alanazi, "Identification and prediction of chronic diseases using machine learning approach," 2022.

[64] K. Gaurav, A. Kumar, P. Singh, A. Kumari, M. Kasar, and T. Suryawanshi, "Human disease prediction using machine learning techniques and real-life parameters," 2023.

[65] R. Alanazi, "Identification and prediction of chronic diseases using machine learning approach," 2022.

[66] M. J. Page, J. E. McKenzie, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, et al., "The PRISMA 2020 statement: an updated guideline for reporting systematic reviews," *BMJ*, vol. 372, art. n71, 2021, doi: 10.1136/bmj.n71.

[67] M. J. Page, D. Moher, P. M. Bossuyt, I. Boutron, T. C. Hoffmann, C. D. Mulrow, et al., "PRISMA 2020 explanation and elaboration: updated guidance and exemplars for reporting systematic reviews," *BMJ*, vol. 372, art. n160, 2021, doi: 10.1136/bmj.n160.

[68] PRISMA Executive, "PRISMA 2020 flow diagram," *PRISMA Statement*, 2026. [Online]. Available: https://www.prisma-statement.org/prisma-2020-flow-diagram

[69] T. Chen and C. Guestrin, "XGBoost: a scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[70] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: a highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 3146–3154.

[71] S. Ö. Arık and T. Pfister, "TabNet: attentive interpretable tabular learning," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 8, 2021, pp. 6679–6687, doi: 10.1609/aaai.v35i8.16826.

[72] P. Patil, "Disease symptom prediction dataset," *Kaggle*, 2020. [Online]. Available: https://www.kaggle.com/datasets/itachi9604/disease-symptom-description-dataset

[73] A. Nair, "Community healthcare MultiSymptoms disease diagnostic dataset," *Kaggle*. [Online]. Available: https://www.kaggle.com/datasets/arjunnairedu

[74] Shobhit043, "Diseases and their symptoms," *Kaggle*. [Online]. Available: https://www.kaggle.com/datasets/shobhit043

[75] Dhivyesh R. K., "Disease-symptom dataset (final augmented)," *Kaggle*. [Online]. Available: https://www.kaggle.com/datasets/dhivyeshrk/diseases-and-symptoms-dataset

[76] D. T. Truong, S. V. Nguyen, and K. T. Nguyen, "Decision support system for drug recognition and disease diagnosis," 2025.

[77] M. Addou, E. B. Mermri, and M. Gabli, "From exponential to efficient: a novel matrix-based framework for scalable medical diagnosis," *BioMedInformatics*, vol. 5, no. 4, art. 68, 2025.

[78] A. Yernagula, K. K. Panigrahi, O. Torkadi, and S. Kulkarni, "Design and evaluation of a multilingual AI-driven healthcare support system," *Int. J. Creative Res. Thoughts (IJCRT)*, 2026.

[79] G. Singh, K. Gandhi, and K. J. Kumar, "Arogya AI: a dual-engine explainable hybrid intelligence system and personalized Ayurvedic health assistant," 2025.

[80] N. Aruna, S. V. Guptha, C. S. Shanmuganandha, and M. Thirumurugan, "MediBot: an AI-powered multi-disease diagnostic web application," *Int. J. Multidiscip. Res. (IJFMR)*, vol. 7, no. 6, 2025, doi: 10.36948/ijfmr.2025.v07i06.62916.

[81] Google LLC, "Flutter: build apps for any screen." [Online]. Available: https://flutter.dev. [Accessed: Jul. 17, 2026].

[82] S. Ramírez, "FastAPI," *FastAPI Documentation*. [Online]. Available: https://fastapi.tiangolo.com. [Accessed: Jul. 17, 2026].

[83] F. Pedregosa et al., "Scikit-learn: machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[84] D. W. Hosmer, S. Lemeshow, and R. X. Sturdivant, *Applied Logistic Regression*, 3rd ed. Hoboken, NJ, USA: Wiley, 2013.

[85] Render Services, Inc., "Render: cloud application hosting for developers." [Online]. Available: https://render.com. [Accessed: Jul. 17, 2026].

[86] Google LLC, "Material Design guidelines." [Online]. Available: https://m3.material.io. [Accessed: Jul. 17, 2026].

Appendix

Appendix A: Model Hyperparameter Configuration

Model	Core configuration	Stopping rule	Hardware
XGBoost (multi:softprob)	n_estimators = 300, eval_metric = mlogloss	Early stopping, patience = 20, best iteration = 35	Google Colab (GPU)
LightGBM (multiclass)	n_estimators = 300, learning_rate = 0.1, num_leaves = 63	Early stopping, patience = 20, best iteration = 1	Google Colab (CPU)
TabNet (n_d = 32, n_a = 32, n_steps = 4)	Batch size = 1024, virtual batch size = 128	Max epochs = 100, patience = 15, best epoch = 32	Google Colab (GPU)
MLP (512-256-128, BatchNorm)	Dropout = 0.35/0.30/0.20, Adam, lr = 1e-3, batch size = 1024	Max epochs = 80, patience = 8 on validation top-1 accuracy	Google Colab (NVIDIA T4 GPU)

Table A.1. Model-specific configuration and stopping behaviour for the four models compared in Chapter IV.

Appendix B: Classification Report Summary (Macro/Weighted Averages)

Note: Full per-class classification reports across all 512 disease classes were generated during model evaluation but are not reproduced here in full due to length, the summary rows below reflect the same evaluation runs reported in Chapter IV.

XGBoost

	Precision	**Recall**	**F1-score**	**Support**
Accuracy	0.8019	0.8019	0.8019	0.8019
Macro avg	0.7752	0.7574	0.7588	28,178
Weighted avg	0.8081	0.8019	0.8030	28,178

LightGBM

	Precision	Recall	F1-score	Support
Accuracy	0.6501	0.6501	0.6501	0.6501
Macro avg	0.5386	0.6400	0.5371	28,178
Weighted avg	0.7513	0.6501	0.6804	28,178

TabNet

	Precision	Recall	F1-score	Support
Accuracy	0.8165	0.8165	0.8165	0.8165
Macro avg	0.8159	0.7776	0.7801	28,178
Weighted avg	0.8404	0.8165	0.8188	28,178

Multilayer Perceptron

	Precision	Recall	F1-score	Support
Accuracy	0.8277	0.8277	0.8277	0.8277
Macro avg	0.8253	0.7839	0.7923	28,178
Weighted avg	0.8452	0.8277	0.8296	28,178

Table B.1–B.4. Classification report summary rows for XGBoost, LightGBM, TabNet, and MLP respectively (single train/test split, prior to cross-validation).

Appendix C: Full Comparative Study Table

Source	Model	Reported Accuracy	Dataset Scale (Disease Classes)
This thesis - LightGBM	LightGBM	65.01%	512
This thesis - XGBoost	XGBoost	80.19%	512
This thesis - MLP	Dense MLP	82.77%	512
This thesis - TabNet	TabNet	82.84%	512
Truong et al. [76] - Naive Bayes baseline	Naive Bayes	83.65%	773 (DS-4-equivalent)

Truong et al. [76] - Random Forest baseline	Random Forest	84.23%	773 (DS-4-equivalent)
Truong et al. [76] - DNN	Deep Neural Network	86.74%	773 (DS-4-equivalent)
Addou et al. [77] - MARS	MARS (matrix-based CBR)	87.34%	721 (largest of four evaluation sets)
Multilingual system [78]	Ensemble (RF+SVM+DNN)	88.2%	Not stated precisely
Arogya AI [79]	Random Forest (TF-IDF+SMOTE)	99.8%	Not stated
MediBot [80]	LR / RF / NB / KNN / XGBoost	Not stated as a single figure	773 (DS-4-equivalent)

Table C.1. Consolidated comparison of this thesis's results against five previously published symptom-disease prediction studies, ordered from lowest to highest reported accuracy.

Appendix D: Source Datasets Used in Master Dataset Construction

Dataset	Source	Records	Disease Classes	Symptom Features	Nature / License
DS-1	*Disease Symptom Prediction Dataset*, P. Patil (itachi9604), Kaggle	4,920	41	131	Curated benchmark; Open Database License (ODbL)
DS-2	*Community Healthcare MultiSymptoms Disease Diagnostic Dataset*, A. Nair (arjunnairedu), Kaggle	100,000	46	174	Synthetic/simulated; CC BY-SA 4.0
DS-3	*Diseases and their Symptoms*, Shobhit043,	96,088	100	230	Curated benchmark; Apache 2.0

	Kaggle				
DS-4	*Disease-Symptom Dataset (Final Augmented)*, Dhivyesh R. K., Kaggle	246,945 raw (189,647 after removing 57,298 duplicates)	773	377	Synthetic/augmented; World Bank Dataset Terms of Use

Table D.1. Characteristics of the four source datasets merged to construct the Master Dataset (DS-1 through DS-3), with DS-4 additionally retained as the standalone large-scale dataset used for the second experimental track described in Chapter III.

Note: DS-1 was treated as authoritative during merging — all its records were retained with full fidelity, and DS-2/DS-3/DS-4 were standardised to match its disease and symptom naming conventions. None of DS-1's supplementary files (severity weights, descriptions, precautions, medications) contributed additional training rows; they populate the application's knowledge base described in Chapter III.